密　　级：_____

加密论文编号：_____

网络设备运行状态监控系统的设计与实现

学　　号：M202340086

作　　者：李根

专 业 名 称：软件工程

2025 年 12 月 20 日

115.25.60.50

网络设备运行状态监控系统的设计与实现

Design and Implementation of a Network Device Operational Status Monitoring System

研究生：李根

指导教师：周贤伟

北京科技大学数理学院

北京 100083，中国

Master Degree Candidate：Li Gen

Supervisor：Zhou Xianwei

School of Mathematics and Physics

University of Science and Technology Beijing

30 Xueyuan Road，Haidian District

Beijing 100083，P.R.CHINA

115.25.60.50

中图分类号：　　学校代码：10008
U　D　C：　　密　　级：公开

北京科技大学硕士学位论文

论文题目：网络设备运行状态监控系统的设计与实现

作者：李根

指 导 教 师：周贤伟 **单位：**数理学院 **职称：**教授

指导小组成员：_____ **单位：**_____ **职称：**_____

_____ **单位：**_____ **职称：**_____

论文提交日期：2025 年 12 月 20 日

学位授予单位：北 京 科 技 大 学

115.25.60.50

摘　要

随着数字化转型不断推进，数据中心网络设备数量日趋增多，网络设备运行状态智能监控能力逐步成为保障网络安全与业务连续性的关键指标，传统监控系统在复杂网络环境下普遍存在数据采集延迟高、固定阈值难以适应动态变化、系统智能化不足等问题，基于上述问题，本文设计并实现了一套网络设备运行状态智能监控系统。

提出基于异构协议融合的网络设备数据采集方法，结合 SNMP、ICMP、Syslog、SNMP Trap 和 Telemetry 等多种网络管理协议，实现多源数据的协同采集与融合处理，充分利用不同协议在数据类型、采集机制及实时性能方面的互补优势，通过模块化、多层次的协议融合与智能调度架构实现动态协议选择，将平均采集延迟降低约 48.7%。该方法在采集效率、数据准确性等方面均优于传统单协议采集模式，为监控系统研究提供数据基础和技术支撑。

提出基于知识推理的多算法融合自适应动态基线阈值的算法（KR-MFA-DBT），在融合统计分析、平滑拟合与补偿修正多算法的基础上引入大模型知识推理机制，实现阈值的自适应与动态调整，该算法较传统的多算法融合动态基线方法 F1 提升约 9.5%，误报率下降约 45%，有效提升异常检测的准确性与鲁棒性。此外，该方法基于大模型输出的推理日志显著提升了基线参数调整的可解释性。

基于上述两种方法的创新，本文设计了一套从数据采集、异常检测、告警提醒到可视化展示的全流程网络设备运行状态监控系统，实现了对网络设备运行状态的实时、精准、自适应监控。系统采用消息中间件与时序数据库实现高效的数据流转与存储，在多场景、多负载条件下均保持了较高的稳定性与可用性，该系统为智能化运维提供了具备实践可行性的技术框架和实现参考，对构建自主化、认知型网络运维体系具有重要的研究意义与推广价值。

关键词： 网络设备监控；多协议融合；自适应动态基线；智能运维

Design and Implementation of a Network Device Operational Status Monitoring System

ABSTRACT

With the continuous advancement of digital transformation, the scale of data center networks has expanded rapidly, making intelligent monitoring of network device operational states a critical capability for ensuring network security and business continuity. Traditional monitoring systems often suffer from high data collection latency, static thresholds that fail to adapt to dynamic environments, and limited intelligent processing, particularly in heterogeneous and large-scale network scenarios. In response to these challenges, this study designs and implements an intelligent monitoring system for network device operational status.

This work first proposes a heterogeneous-protocol-fusion data acquisition method that integrates multiple network management protocols, including SNMP, ICMP, Syslog, SNMP Trap, and Telemetry. By leveraging the complementary advantages of different protocols in terms of data types, acquisition mechanisms, and real-time capabilities, the method enables collaborative multi-source data collection and fusion processing. A modular and multi-layer protocol-fusion architecture with intelligent scheduling supports dynamic protocol selection, reducing average data collection latency by approximately 48.7%. Compared with traditional single-protocol approaches, this method delivers significant improvements in collection efficiency and data accuracy, providing a solid foundation for intelligent monitoring research and applications.

Furthermore, this study introduces KR-MFA-DBT, a large-model-enhanced multi-algorithm fusion method for adaptive dynamic baseline thresholding. By combining statistical analysis, smoothing fitting, and compensatory correction with a knowledge-reasoning mechanism powered by large models, the algorithm achieves adaptive and dynamically adjustable thresholds. Experimental results show that KR-MFA-DBT improves F1-score by about 9.5% and reduces the false-alarm rate by approximately 45% compared with traditional multi-algorithm dynamic baseline methods. Additionally, reasoning logs generated by the large

model significantly enhance interpretability in baseline parameter adjustments.

Building upon these two methodological innovations, the study develops an end-to-end intelligent monitoring system covering data acquisition, anomaly detection, alerting, and visualization. The system employs message-oriented middleware and time-series databases to support efficient data transmission and storage, maintaining high stability and availability across diverse scenarios and workloads. Overall, this work provides a practical and extensible framework for intelligent network operations and offers valuable reference for constructing autonomous and cognitive network operation systems.

Key Words: network device monitoring; multi-protocol fusion; adaptive dynamic baseline; intelligent operation and maintenance

目　录

摘　要 I
ABSTRACT II
符号清单 VIII
1 绪论 1
1.1 研究背景及意义 1
1.1.1 研究背景 1
1.1.2 目的及意义 1
1.2 国内外研究现状 2
1.2.1 网络设备数据采集任务研究现状 2
1.2.2 阈值监控技术研究现状 3
1.3 本文研究内容及主要贡献 4
1.4 本文组织结构 5
2 网络设备运行状态智能监控系统需求分析与总体设计 6
2.1 引言 6
2.2 系统需求分析 6
2.2.1 系统总体需求分析 6
2.2.2 系统功能需求分析 7
2.2.3 系统性能需求分析 7
2.2.4 系统安全性与高可用性需求分析 8
2.2.5 系统扩展性与兼容性需求分析 9
2.3 系统架构设计 9
2.3.1 分层架构设计 10
2.3.2 分层协作与数据流动 12
2.4 系统功能模块设计 13
2.5 系统安全性与高可用性设计 13
2.6 系统可扩展性与兼容性设计 13
2.7 本章小结 14
3 基于异构协议融合的网络设备数据采集方法 16
3.1 引言 16
3.2 方法设计 16
3.2.1 数据采集层 18

3.2.2 数据处理与融合层 18
3.2.3 智能调度与协议选择层 20
3.3 实验与分析 23
3.3.1 评价指标 23
3.3.2 实验环境及数据集 24
3.3.3 对比方法 25
3.3.4 实验方案 25
3.3.5 实验结果及分析 25
3.4 本章小结 27
4 基于知识推理的多算法融合自适应动态基线阈值算法 29
4.1 引言 29
4.2 任务定义 29
4.2.1 问题描述 29
4.2.2 数学建模 30
4.3 算法设计 31
4.3.1 算法总体设计 31
4.3.2 创新性体现 37
4.3.3 算法正确性与收敛性证明 37
4.3.4 时间复杂度分析 37
4.4 实验与分析 38
4.4.1 评价标准 38
4.4.2 实验环境与数据集 39
4.4.3 对比算法 39
4.4.4 实验结果与分析 40
4.5 本章小结 43
5 网络设备运行状态智能监控系统实现与测试 45
5.1 引言 45
5.2 系统技术架构 45
5.3 系统功能模块实现 46
5.3.1 监控展示模块实现 46
5.3.2 设备管理模块实现 48
5.3.3 任务管理模块实现 50
5.3.4 规则管理模块实现 52
5.3.5 Ping 采集模块实现 53

5.3.6 智能告警模块实现 54
5.3.7 日志管理模块实现 55
5.3.8 性能报表模块实现 55
5.4 系统测试 56
5.4.1 功能测试 56
5.4.2 非功能测试 57
5.5 本章小结 60
6 结论与展望 62
6.1 总结 62
6.2 展望 63
参考文献 65
致谢 71
作者简历及在学研究成果 73
独创性说明 75
关于论文使用授权的说明 77
学位论文数据集 79

符号清单

符号	含义说明
Ω	所有设备或数据点的集合
Y_m	设备 m 的真实值
$\hat{Y}_m$	模型采集的设备 m 数据值
$t_{send,i}$ $t_{recv,i}$	第 i 次采集请求的发送和接收时间
P_i	网络状态变化的概率
ΔS_i	切换后采集效率提升的比例
D	网络设备在监控周期内采集的数据序列
B_t^{up} B_t^{low}	时刻 t 的上下阈值
θ_t	时间 t 的算法权重参数集
ω	时间窗口大小
x_t	时刻 t 的指标数值
$B_t^{3\sigma}$	传统统计型基线
B_t^{smooth}	趋势平滑基线
B_t^{comp}	补偿修正基线
α_t	传统统计型基线权重系数
β_t	趋势平滑基线权重系数
γ_t	补偿修正基线权重系数
$\mathcal{L}(W_t)$	目标损失函数
F_t	方差、波动率、趋势因子等实时统计特征信息
C_t	协议类型、设备类别、网络层级等设备上下文信息
H_t	异常类型、误报率、阈值漂移趋势等历史监控知识
Φ_{LLM}	知识推理映射函数
Ψ_{Rule}	权重约束逻辑推理函数
ΔW_t	权重变化率
ΔB_t	基线波动率

1 绪论

1.1 研究背景及意义

1.1.1 研究背景

随着云化技术的不断深化、数据中心规模逐渐扩张，网络设备种类日趋多样化，网络运行环境变得愈发复杂。路由器、交换机、防火墙及各类边缘设备在承载关键业务的同时，也面临着可用性、延迟率和吞吐率要求不断提高的标准和要求，因此对网络设备运行状态的精确和及时、有效地监控成为保障业务连续性的核心工作。

基于传统 SNMP/ICMP 网络协议的轮询式监控数据采集方法，在小规模或静态场景下被广泛使用，但在高并发、高频变以及更大规模的网络环境中，单一轮询机制延迟高、负载大、观测盲区明显、跨厂商兼容性差等短板也逐渐显现。随着技术的不断发展和丰富，Telemetry、Syslog、SNMP Trap 等主动上报和事件驱动的数据采集机制[1]因其能提供更细粒度和更低延迟的数据流逐步被引入运维体系，这为构建实时性更强的监控平台提供了更高的技术可能性[2]。网络设备监控数据存在高维、强时序与多周期的特征，因此，单纯依靠传统的人工设定或固定阈值难以维持长期稳定的告警精度，误报和漏报概率逐渐增大[3]，亟须将多算法协同、实时处理与自适应阈值调整等方法融入监控体系当中。

综上，构建一套基于多协议融合和智能算法驱动的网络设备运行状态监控系统，通过多协议融合实现多源数据的协同采集与语义统一，依托多算法融合与智能化阈值实现对时序特征和异常模式的自适应建模，从而对提升网络运维自动化水平和智能化能力具有重要的现实意义与研究价值。

1.1.2 目的及意义

本研究旨在设计并实现一套面向复杂网络环境的智能化网络设备运行状态监控系统，通过融合基于异构协议融合的网络设备运行状态数据采集方法和基于知识推理的多算法融合自适应动态基线阈值方法[4]，提升数据采集的完整性与实时响应能力，实现智能化的阈值自学习与异常检测，构建高可用

与可扩展的监控体系结构，从而推动智能运维的自主化与认知化发展。

1.2 国内外研究现状

1.2.1 网络设备数据采集任务研究现状

数据采集是监控系统能够顺利运行的基础，其重点在于如何合理地使用采集协议、增强数据处理性能并融合多源数据等方面。

20 世纪 90 年代，早期的网络管理系统采用 SNMP[5-6]和 ICMP[7]进行设备运行状态和网络连通性的监测，SNMP 协议主要通过轮询的方式定期采集设备的性能数据和状态信息，ICMP 协议主要用于链路质量监测，上述两种网络协议的监控机制简单有效，但在大规模网络环境下可能会带来高延迟和大量的带宽消耗[8]。随着网络规模的逐渐扩大，Syslog[9]与 SNMP Trap[10]等事件驱动型采集协议逐渐取代了轮询式的采集方案，从而提升了告警响应速度与数据实时性。21 世纪后，网络设备数据采集逐步转向实时、高频率的数据收集和分析，Telemetry 协议[11]在网络设备数据采集任务中具有重要的研究价值，区别于传统的 SNMP 轮询机制，Telemetry[12]能够有效支持设备端主动推送高频度和结构化的数据流，从而降低采集延迟，面对复杂且大规模的网络环境时，Telemetry 能够有效提升数据采集的实时性，对运维和自动化的控制起到了重要作用[13]。

随着数字化进程的不断加快，网络环境日趋复杂，单一协议采集的局限性逐渐显现，多协议融合的方式逐渐在实践中开始应用，通过整合多种协议可以提供更全面、更精确的网络监控能力，但同时也面临着如何高效地从不同来源的数据中提取有效的监控数据信息，从而提升数据采集的准确性和响应能力。经研究，基于查询驱动或模型驱动的遥测架构可以满足多源数据的统一建模与流式处理，从而有效地协调不同数据源之间的数据语义与传输开销，在保证可观测性的同时，实现高效的分布式采集与聚合[14-15]。Shir Landau-Feibish 等的研究探索了在带宽与计算资源受限场景下通过紧凑数据结构与流处理管道可以显著降低传输成本并提升系统吞吐率，从而为大规模部署提供了可行的工程路径[16]。随着流数据处理技术的发展，基于 Apache Kafka 和 Flink 等大数据平台的网络数据采集系统能够实时地处理大规模数据流，进一步提高了网络设备数据采集和分析的效率[17-18]。

综上，网络设备数据采集的方法经历了从传统单一协议轮询采集向多协议协同与智能采集的深刻演进。早期的采集方式主要依赖 SNMP 等独立协议，

存在数据维度有限、实时性不足、设备兼容性差等问题。随着网络规模与复杂性的持续提升，多协议融合采集逐渐成为主流，通过整合各类管理协议与接口数据，实现了对网络状态更全面、更高效的感知。当前，该领域的研究重点主要集中在三个方面：一是优化采集性能，通过改进数据获取机制、压缩传输负载、提升处理并发度，以降低对设备和带宽的影响；二是提升协议互融能力，设计统一的协议适配与转换框架，增强对不同厂商、不同层级设备数据的兼容性与解释能力；三是增强系统扩展性，构建弹性可伸缩的采集架构，支持动态接入新型设备与采集协议。在此基础上，本文在采集策略自适应调整以及多源数据融合一致性等方面展开进一步的探索。

1.2.2 阈值监控技术研究现状

阈值监控的方案经历了从静态阈值到动态基线阈值与智能化分析的演变过程。早期的网络监控系统主要采用固定阈值的监控策略，运维人员根据经验为网络设备的 CPU 利用率、链路带宽、错误包率等各项监控指标设定固定的上下限阈值，当监控指标超出预设范围后即触发告警，这种方式的实现简单、直接，但由于阈值缺乏自适应能力，在面对不同时间周期、业务负载或设备类型变化时，容易产生大量误报与漏报，且阈值维护人工干预因素较大，难以构建出持续性强、适用性好的监控体系，人工维护成本相对较高[19]。

因此，为解决传统阈值策略的局限，实践中逐渐引入了基线阈值的概念，传统的基线模型通常采用统计的方法进行计算，计算过程一般可分为 4 个步骤，首先为历史数据样本的采集，第二个步骤为数据样本降噪处理，第三个步骤为选取合适的统计指标并进行基线阈值的计算，第四个步骤为设定基线阈值。IBM 在早期的研究中提出了基于流量基线建模的异常检测方法，可根据历史特征动态调整告警阈值。随后，BMC、IBM 等早期运维平台以历史平均值、标准差及百分位为基础数据，构建了基线阈值监控机制[20-21]，大幅缓解了人工配置的压力[22]，但这一阶段的基线仍主要依赖离线计算的结果，更新频率较低，难以应对网络运行环境的快速波动。

进入云计算与智能运维时代后，监控数据呈现出明显的高维度、强时序与多周期性的特征，传统基线方法在动态性和精度上均存在不足。于是，动态基线的概念随之应运而生，动态基线方法是指通过对指标历史数据的持续建模，结合时间序列特征、周期规律与异常数据的分布，自适应地更新基线阈值[23]，在不同时间段、日期类型或业务状态下动态调整正常区间[24-25]。Yu Y 等学者提出利用滑动窗口方式提取均值、标准差等统计特征并据此构造异

常检测阈值[26]，Isaac 等提出的 ATH（Adaptive Thresholding Heuristic）算法通过局部分布与季节性建模实现 KPI 异常自适应检测[27]，X. Yang 等人提出将阈值设定建模为马尔可夫决策过程，采用深度 Q - 网络（DQN）[28]进行阈值自学习优化，并使用 Renyi 熵改进 EWMA 模型，在固定时间窗口内计算阈值[29]。近年来，基于不确定性量化的动态阈值估计方法、模型置信度调整阈值敏感度，增强了系统在噪声数据下的稳定性，但随着大语言模型在自然语言处理、推理等方面表现出的泛化能力，研究者们开始将应用于时间序列、日志分析、异常检测等任务[30-31]，这为更智能的基线构建提供了可能。

综上所述，基线阈值监控技术的发展脉络清晰地呈现出从经验驱动到数据智能驱动的深刻转变。其技术路径始于依赖人工经验的固定阈值设定，虽在早期阶段实现了一定程度的异常检测，但存在适应性差、误报率高、维护成本大等局限。随着大数据与机器学习技术的成熟，该领域逐步演进至动态基线方法阶段：系统能够基于历史时间序列的内在规律、不同业务场景的负载特征，以及实时数据流的分布模式，自动生成并持续更新阈值基线。这一转变不仅是技术工具的升级，更是监控范式的革新，降低了运维对人工经验的依赖，并为后续的智能运维体系构建了坚实的算法基础与理论框架。

在上述演进成果的基础上，本文提出了一种基于知识推理的多算法融合自适应动态基线阈值方法。该方法以动态基线理论为根基，进一步引入了大语言模型（LLM）作为知识推理与语义理解的核心引擎。通过 LLM 对运维知识、场景上下文与历史事件的深层理解与逻辑推断能力，本方法能够更智能地融合统计特征和时间序列，实现对复杂业务场景与多变网络环境的精准建模，不仅增强了基线阈值对突变模式与渐进式异常的识别灵敏度，也大幅提升了系统在不同运维阶段的自主适应与决策能力。

1.3 本文研究内容及主要贡献

本文以复杂网络环境下网络设备运行状态智能监控系统为研究对象，围绕运维中数据采集效率低、阈值告警精度不足等关键问题，提出了一套融合异构协议数据采集与知识推理多算法自适应动态阈值的系统化解决方案。本文的研究内容主要包括以下几个方面：

(1) 基于异构协议融合的网络设备数据采集方法

针对不同网络设备和监控场景的异构特性，构建了基于 SNMP、ICMP、Syslog、SNMP Trap 与 Telemetry 等多协议融合的数据采集框架，提出通过任务调度与协议选择机制，实现采集效率、数据实时性与准确性的平衡。

(2) 基于知识推理的多算法融合自适应动态基线阈值方法

为解决传统静态阈值告警方法的低精度问题，本文提出基于统计方法、平滑拟合法与补偿算法的多算法融合机制，并引入知识推理与规则约束，通过设计权重自适应优化模型 KR-MFA-DBT，显著提升了系统的智能性与监控阈值的自学习能力。

(3) 网络设备运行状态智能监控系统设计与实现

构建分层、模块化的系统架构，设计数据采集、融合处理、智能告警、可视化展示等核心模块，形成端到端的监控体系，实现高并发、高可靠性与可扩展部署。

(4) 系统验证与性能测试

通过在真实网络环境下的实验与多节点部署测试，从准确率、误报率、响应时间与系统可用性等指标进行了综合评估，验证了系统在大规模设备场景中的稳定性与有效性。

1.4 本文组织结构

本文共分为六章，主要内容和安排如下：

第一章是绪论。主要介绍研究背景、研究意义、国内外研究现状及存在问题，明确本文的研究目标、主要内容与创新点。

第二章是系统需求分析与总体架构设计。从总体需求、功能需求、性能需求、安全与高可用性需求以及系统扩展性需求等方面进行系统性需求分析，并对系统总体架构设计、模块功能设计、模块间协作关系进行系统化阐述。

第三章是基于异构协议融合的网络设备数据采集方法设计。阐述异构协议数据采集方法的设计思路，详细介绍数据采集、数据处理与融合、智能调度与协议选择等关键设计。

第四章是基于知识推理的多算法融合自适应动态基线阈值方法研究。提出多算法融合与知识推理驱动的动态阈值参数自适应模型，并通过实验验证算法的有效性。

第五章为系统实现与测试。阐述了系统的技术实现和功能模块实现，并从功能、性能、安全性与高可用性、可扩展性与兼容性四个方面给出了系统实现的全面验证结果。

第六章是结论与展望。总结本文的研究成果与意义，分析研究中的不足，并展望未来的优化方向。

2 网络设备运行状态智能监控系统需求分析与总体设计

2.1 引言

随着网络规模的持续扩张和数据中心业务形态的不断演进，监控系统在实时性、准确性、可扩展性及安全性方面的要求显著提升。在系统构建阶段开展全面而严谨的需求分析与架构规划，不仅是系统后续实现的必要前置条件，也是确保整体系统稳定性与可演进性的关键环节。

为构建面向复杂网络环境的智能化监控体系，本章将从系统设计目标出发，对网络设备智能监控系统的需求、原则与总体架构设计进行系统性的分析。重点围绕系统功能需求、性能指标、安全性与高可用性要求、扩展性与兼容性要求深入分析，进而提出符合现代数据中心监控特征的系统总体架构。通过对各层级设计理念、模块间协作关系以及关键技术组件的系统化阐述，为后续系统实现与性能验证提供理论依据与结构化指引，从架构层面奠定系统实现的基础性框架。

2.2 系统需求分析

本节主要对网络设备运行状态智能监控系统的需求进行系统化、层次化分析与介绍，明确系统建设目标与功能定位，为后续系统架构设计与功能模块实现提供理论依据与设计指导。从系统总体需求、系统功能需求、系统性能需求、系统安全性与高可用性需求、系统扩展性与兼容性需求五个方面对网络设备运行状态智能监控系统进行全面需求分析，为后续系统设计和实现奠定基础。

2.2.1 系统总体需求分析

网络设备运行状态智能监控系统的总体需求是构建一个能够实时监控、分析和预测网络设备运行状态的在线平台[32]，该系统能够从路由器、交换机、防火墙、服务器等多种网络设备中自动采集数据，通过数据处理与分析对网络设备进行实时性能监控与故障检测，并将监控结果进行可视化展示，及时发现网络设备运行的潜在问题[33]。此外，系统还应具备告警与通知机制，基于设备运行状况和分析结果自动生成告警，并通过邮件、短信等多种方式通

知运维人员。系统需要具备为用户提供可视化展示界面的能力，清晰地展示设备运行状态、性能趋势以及告警信息。系统还应具备良好的扩展性与兼容性，能够支持新的设备类型和监控协议的接入，同时确保与现有网络架构和管理工具的兼容。

2.2.2 系统功能需求分析

系统功能需求是基于上述总体需求展开的，包括数据采集功能、数据存储功能、数据处理与分析功能、告警与通知功能、可视化展示功能和用户管理功能。具体功能需求见表 2-1。

表 2-1 网络设备运行状态智能监控系统功能需求

Tab.2-1 Functional Requirements of the Intelligent Monitoring System for the Operating Status of Network Equipment

功能需求	简介
数据采集功能	协议支持：支持多种网络协议采集设备的性能数据，定期或实时从网络设备中拉取或推送数据，保证数据的完整性和准确性 数据过滤与清洗：对采集数据进行清洗，去除无效和错误数据
数据存储功能	数据库支持：存储采集数据，存储方式应考虑高效性和扩展性，存储结构需支持高频次实时数据更新以及历史数据长期存储 数据安全与备份：保证数据安全性并定期备份，防止丢失和损坏
数据处理与分析功能	自适应动态基线：基于大模型知识推理实时调整基线阈值，动态更新 故障预测与异常检测：检测网络设备异常行为，提前预警可能的故障
告警与通知功能	告警策略：基于实时数据和动态阈值，实现设备性能异常监测预警 告警内容：支持对设备故障、性能异常、资源瓶颈等内容进行告警 告警通知：支持邮件、短信、告警看板即时推送等多种通知方式
可视化展示功能	多种可视化组件：提供 CPU 负载、内存使用率、流量变化等性能指标图表，告警信息列表，设备运行健康状态面板等可视化展示组件 数据展示与查询分析：展示实时监控数据和历史趋势，支持对特定时间段内的设备数据进行查询和分析
用户管理功能	权限管理：支持多角色用户权限管理，不同角色实现权限隔离 日志审计：记录系统操作日志，支持日志审计，确保安全性和可追溯性

2.2.3 系统性能需求分析

系统性能是保证系统稳定性、可靠性和高效性的关键指标，也决定了最终用户的使用体验。通过对性能需求的合理规划，可以确保系统能够高效地处理大量设备数据，及时发现潜在问题，快速响应故障预警，保证网络环境的稳定运行。良好的性能设计可以为后续系统扩展和升级提供保障，降低后

期维护和优化成本。本系统的性能需求需要考虑响应时间、数据处理能力、数据存储容量等。系统在性能方面的详细需求见表 2-2。

表 2-2 网络设备运行状态智能监控系统性能需求

Tab.2-2 Performance Requirements for the Intelligent Monitoring System of Network Device Operating Status

性能需求	简介
响应时间	系统应能够实时响应设备状态查询和告警通知请求。在大规模网络环境下，响应时间不超过 2 秒
数据处理能力	系统应能够处理高频次设备数据采集，具备每秒处理数千条设备数据的能力
数据存储容量	系统需支持海量数据存储，能够存储至少一年以上的设备历史数据，且能根据存储量自动进行数据归档和清理

2.2.4 系统安全性与高可用性需求分析

系统需考虑安全性需求和重要故障发生时能够持续提供服务的高可用性需求。安全性需求包括数据安全、权限管理、日志审计，高可用性需求包括冗余设计、负载均衡和故障恢复。具体需求如下：

(1) 安全性需求

在数据传输阶段，采用 SSL/TLS 等加密协议确保通信过程的机密性与完整性。在数据存储阶段，应实施加密存储与访问控制机制，防止未经授权的访问与数据泄露。

系统需构建基于角色的访问控制机制，严格限制不同角色用户的访问范围，确保用户仅能操作其授权范围内的数据资源，从而防止数据的越权访问与篡改。

系统应具备完备的日志审计功能，对用户操作及系统关键事件进行全程记录和留痕，以支持事后安全审计、问题追溯与合规性检查。

(2) 高可用性需求

系统数据库、数据采集模块、分析引擎等关键组件应采用冗余部署方案，以消除单点故障风险，确保在局部组件发生故障时系统仍能持续提供服务。

系统应通过负载均衡技术将用户请求与数据流量合理分配至多个服务器节点，提高资源利用率和并发处理能力，保障系统在高负载情况下的稳定性与响应性能。

系统需建立自动化故障检测与恢复机制，在发生异常或故障时快速切换至备用节点或备份系统，尽可能缩短服务中断时间，提升系统整体可用性与

可靠性。

2.2.5 系统扩展性与兼容性需求分析

系统在设计阶段需充分考虑扩展性与兼容性，以应对未来网络规模的持续增长和多样化技术环境的演进。系统应具备良好的横向扩展能力，能够通过增加硬件资源或云计算资源实现性能的线性提升，从而支持大规模网络设备的高效监控与管理。扩展性需求包括设备扩展、协议扩展与模块扩展[34]，兼容性需求包括跨平台兼容性和多设备兼容性。具体要求如下：

(1) 扩展性需求

系统应支持网络设备的动态接入与自动发现机制，能够在网络规模扩大或新增设备部署时，实现新设备的无缝集成与快速注册，保证系统在大规模场景下的持续监控能力。

系统须具备多协议融合与兼容能力，支持 SNMP、ICMP、Syslog、Telemetry 等主流网络管理协议的灵活集成，适配不同厂商、不同类型设备的通信规范。同时，系统架构应预留接口以支持未来新协议的引入，确保其在技术演进中的可持续性与适应性。

系统各功能模块应采用高内聚低耦合设计原则，以模块化架构实现功能的独立部署与扩展。系统应支持新增功能模块的平滑集成，如引入新的数据分析算法、外部接口模块或可视化展示组件，从而提升系统的可维护性与灵活性。

(2) 兼容性需求

系统应支持在 Windows、Linux 等不同操作系统平台上稳定运行，确保核心服务和应用组件具备良好的跨平台可移植性与运行一致性，为多样化部署环境提供技术支撑。

系统需支持多类型、多厂商网络设备的统一接入与监控，提供标准化的数据采集接口和协议适配机制，实现异构设备间的数据统一建模与集中管理，从而保证系统在复杂网络环境中的通用性与兼容性。

2.3 系统架构设计

本系统采用分层、解耦、协同、可扩展的总体架构设计思想，将系统功能划分为数据采集层、数据处理与存储层、应用服务层、可视化展示层四个核心层次[35]和监控展示、设备管理、任务管理、Ping 采集、聚合规则管理、

智能告警、日志管理、性能报表八个核心模块[36]。系统通过标准化接口与模块化组件实现各层之间的协作[37]，构建了一个高性能、高可用、可扩展的网络设备运行状态智能监控体系[38]，确保系统高效运行的同时具备良好的可维护性。

系统的总体设计目标包括异构协议融合采集、实时数据处理与聚合、基于动态阈值的智能分析与监控、多层次告警与可视化展示、系统安全与容错设计。基于分层架构思想，整体采用自底向上的功能分层结构，各层间职责明确，通信采用 RESTful API 与 WebSocket 实时推送机制。数据采集层参考本文第三章中基于异构协议融合的网络设备数据采集方法进行设计。应用服务层的智能告警引擎参考本文第四章基于知识推理的多算法融合自适应动态基线阈值方法进行设计。

2.3.1 分层架构设计

2.3.1.1 数据采集层

数据采集层位于系统的底层，负责从各类网络设备中获取实时运行数据与性能指标，是系统的数据源入口。该层采用多协议融合和任务调度机制的设计，实现灵活高效的数据采集。该层的主要功能设计见表 2-3。

表 2-3 数据采集层功能设计

Tab.2-3 Functional Design of the Data Acquisition Layer

功能	简介
异构协议采集	使用本文第三章基于异构协议融合的网络设备数据采集方法，支持 SNMP、ICMP、Syslog、SNMP Trap、Telemetry 等异构协议数据采集
采集任务管理	采用任务调度模块统一管理采集计划，支持周期性任务与即时任务
采集间隔与频率配置	允许用户针对不同设备类型自定义采集频率，保证关键设备数据采样密度
异常数据初步处理	在采集端执行数据格式校验、错误过滤与单位换算，减轻后端处理压力
数据缓存与传输	使用轻量级消息队列 MQ 暂存采集结果，通过异步通信将数据发送至数据处理层

2.3.1.2 数据处理与存储层

数据处理与存储层主要负责对采集的数据进行预处理、聚合计算、标准

化存储和索引管理[39]，该层的主要功能设计见表 2-4。

表 2-4 数据处理与存储层功能设计

Tab.2-4 Functional Design of the Data Processing and Storage Layer

功能	简介
数据清洗与标准化处理	清除重复项与异常值；对时间戳与单位进行统一；转换采集结果为标准化 JSON 格式便于后续存储与分析；使用多线程并行处理技术提升批量数据清洗效率
数据聚合与规则计算	根据聚合规则计算统计结果，支持跨设备聚合；使用滑动窗口算法实现时间序列聚合计算[40]
数据存储体系	采用 Oracle 数据库作为核心存储组件，存储设备状态数据、性能指标数据、日志信息及告警记录，Oracle 的高并发事务处理能力与完善的安全机制能够满足系统对数据一致性与可靠性的要求
日志与告警数据管理	集中接收系统日志、设备 Syslog 及 Trap 信息；使用索引快速查询关键字

2.3.1.3 应用服务层

应用服务层是系统的逻辑核心，集成了任务调度、智能告警、日志分析、聚合规则管理与性能报表生成等关键功能，主要功能设计见表 2-5。

表 2-5 应用服务层功能设计

Tab.2-5 Functional Design of the Application Service Layer

功能	简介
任务管理与调度中心	统一管理采集任务、Ping 任务、聚合任务；调度器采用多线程异步机制，支持任务依赖与优先级；提供任务运行状态监控、失败重试
智能告警引擎	使用基于知识推理的多算法融合动态基线阈值方法监控运行状态；支持风暴告警检测进行短期告警爆发识别；支持告警分级管理
聚合规则管理模块	支持用户定义聚合指标，提供多种聚合函数
日志与审计模块	集中记录系统运行日志与操作日志；具备操作行为审计功能，满足安全合规要求
性能报表模块	自动生成设备性能报表、健康度报告

2.3.1.4 可视化展示层

可视化展示层是系统与用户交互的界面，主要功能是将监控数据、性能变化趋势和告警信息进行直观展示。本层采用前后端分离的设计原则，前端

采用 Vue.js + Element UI + jQuery + ECharts 框架开发，后端使用 Zookeeper 管理分布式一致性与服务注册，前后端部署分离。主要功能见表 2-6。

表 2-6 可视化展示层功能设计

Tab.2-6 Functional Design of the Visualization Presentation Layer

功能	简介
仪表盘式可视化界面	展示 CPU、内存、带宽利用率等核心指标
趋势分析图	显示时间序列变化，支持缩放、拖拽与多维对比
告警总览与事件回溯	展示各级别告警数量及分布，可按时间追溯
性能自监控视图	显示系统本身任务运行情况与资源占用
交互式控制功能	用户可在前端发起任务管理、聚合规则定义、告警确认等操作

2.3.2 分层协作与数据流动

系统通过标准化接口实现各层之间的协作，通过各层交互实现了数据采集、处理、展示、反馈的闭环流转，保证系统高效与实时。分层协作见图 2-1。

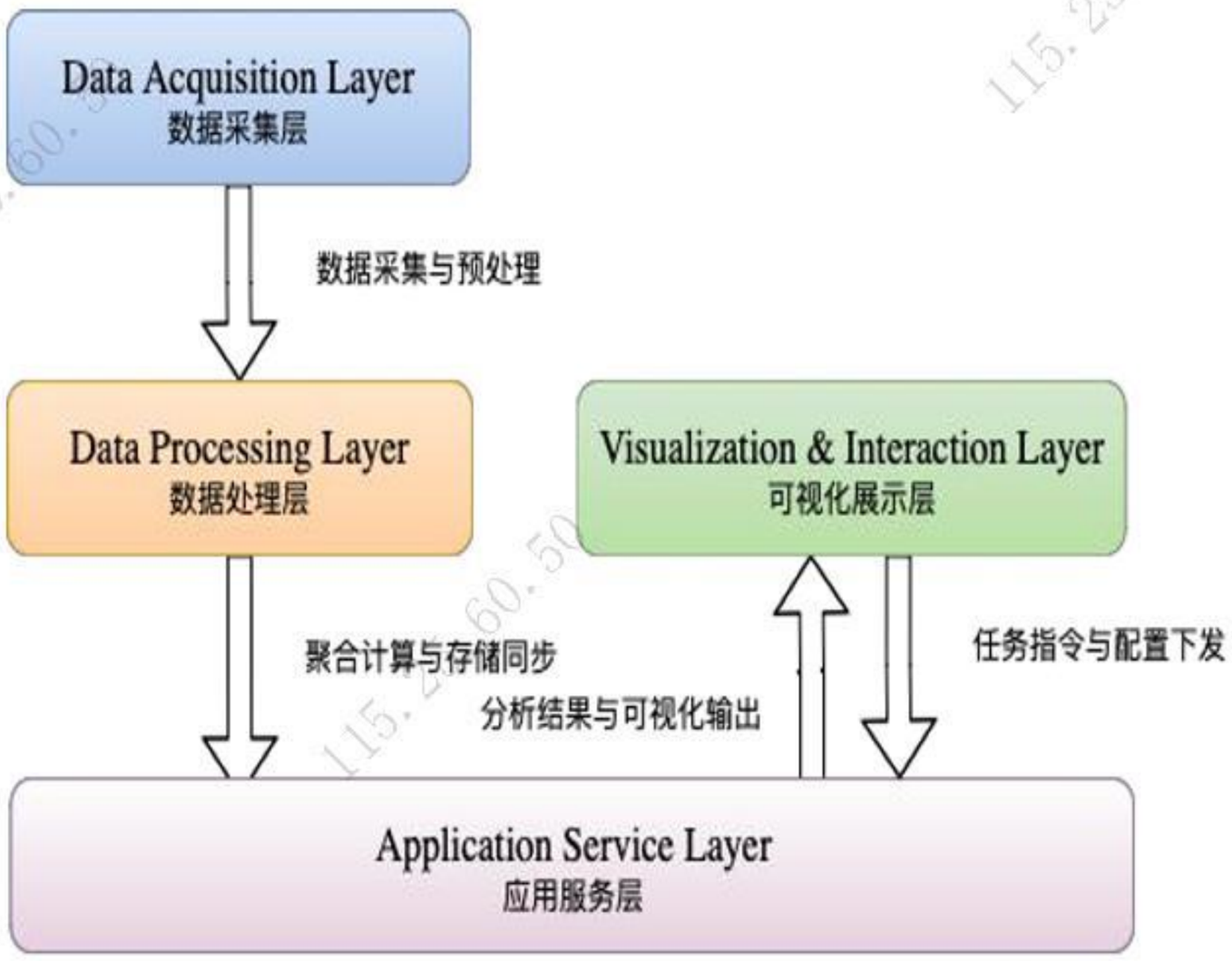


图 2-1 分层协作与数据流动图

Fig.2-1 Layered Collaboration and Data Flow Diagram

2.4 系统功能模块设计

系统由监控展示、设备管理、任务管理、Ping 采集、聚合规则管理、智能告警、日志管理、性能报表八个核心模块组成，各模块之间通过统一业务总线交互，形成完整的数据采集、分析和展示闭环，模块化的设计可以提高系统的扩展性与维护性。系统功能模块交互图见图 2-2。

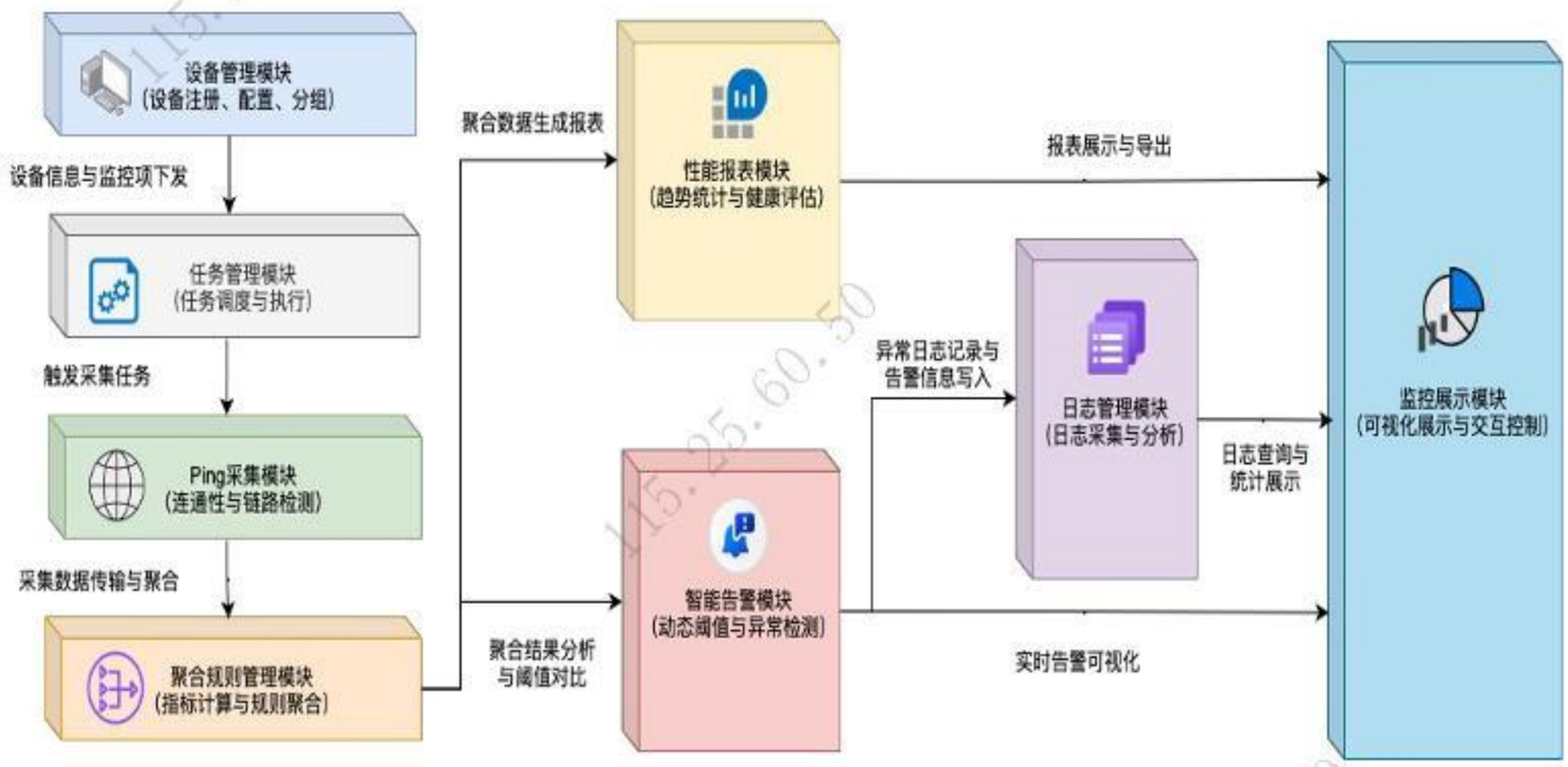


图 2-2 系统功能模块交互图

Fig.2-2 System Functional Module Interaction Diagram

2.5 系统安全性与高可用性设计

在安全性设计方面，本系统使用基于角色的访问控制管理实现用户权限精细化管理[41]，结合 OAuth2.0 认证机制与 HTTPS 加密传输协议确保通信过程的保密性与完整性。此外，系统操作均纳入日志审计，实现可追溯与合规化管理。

在高可用性设计方面，系统通过主备冗余部署、负载均衡调度及任务自动恢复机制，保证核心模块在出现故障时仍可持续运行，同时引入容器化与集群化部署策略，实现服务的动态扩展与故障快速切换；配合定期快照与自动备份策略，确保关键数据的可靠恢复。

整体设计实现了系统在复杂网络环境下的安全可控、稳定高效与持续可用，为网络设备运行状态智能监控系统的长期运行提供了坚实保障。

2.6 系统可扩展性与兼容性设计

为应对网络设备规模增长和业务场景变化，系统设计需要满足可扩展性

和兼容性需求。系统扩展性设计目标是实现设备规模、协议类型及功能模块的可持续扩展，系统兼容性设计目标是确保系统在不同软硬件环境、网络协议及设备类型下均能稳定运行，实现多平台、多厂商、多类型设备的运行状态监控。

系统采用分层模块化设计，各功能模块通过标准化接口与消息中间件实现解耦，在系统需求变化时横向扩展新的功能模块。系统的采集模块支持多协议融合机制，通过协议适配实现对多种网络管理协议的统一封装，当接入新协议或扩展设备类型时无需修改核心逻辑，采用 Oracle 数据库与 Redis 缓存协同架构实现系统数据存储。Oracle 的分区表、索引优化和主从同步机制支持大规模数据的高效读写与长期存储, Redis 的分布式缓存技术支持高并发访问场景下的快速响应，多级存储结构使系统在数据量增长时仍保持稳定的访问性能。

系统可支持麒麟 V10、RedHat Linux 与 CentOS 等多种 Linux 平台，开发框架基于 OpenJDK 与 SpringBoot 实现，具有良好的跨平台特性，从而保证跨平台兼容性，并通过协议适配支持不同厂商设备的数据采集与监控，保证设备的兼容性，通过对外提供 RESTful API 和消息中间件接口，保证接口兼容性。

2.7 本章小结

本章围绕网络设备智能监控系统的设计目标与总体实现思路，对系统需求与架构进行了整体分析与阐述。为系统实现与性能验证奠定基础，为实现系统高可靠性、高兼容性、高安全性与高扩展性提供理论支撑。

本章明确了系统应满足的实时性、可靠性与可扩展性要求，通过分析功能需求、性能需求、安全性与高可用性需求以及扩展性与兼容性需求定义了系统的功能边界与性能指标，为后续系统设计与实现提供了理论依据与工程约束。

在功能设计方面，系统通过异构协议数据采集、数据处理与聚合、动态阈值告警、可视化展示等功能模块构建了完整的监控体系，实现了从数据采集、异常检测、告警提醒到可视化展示的全流程支撑。

在性能设计方面，系统在高并发环境下确保了实时响应与稳定运行，满足了大规模设备监控的处理能力需求。安全与高可用性设计部分重点阐述了系统在加密传输、权限控制、日志审计、冗余部署、负载均衡与故障恢复等方面的策略，以保障系统在关键业务环境中的持续可靠运行。扩展性与兼容

性设计方面，系统采用模块化与松耦合架构，使其能够在设备规模扩大、协议类型增加或功能模块扩展的情况下，保持高效运作与良好的可维护性。通过跨平台与多设备兼容机制，系统可适应异构网络环境下的多厂商设备接入与统一管理，提升了系统的通用性与可移植性。

3 基于异构协议融合的网络设备数据采集方法

3.1 引言

随着网络规模的不断扩大以及新型网络设备的不断更新换代，传统依赖SNMP、ICMP、Syslog 等单一协议的轮询[42]和日志采集方法，已经难以满足高频实时监控和多厂商设备下的兼容性需求，目前，上述方法仍存在延迟高、实时性差、数据处理能力有限等局限性。

因此，多协议融合的数据采集方法应运而生，通过整合 SNMP、ICMP、Syslog、SNMP Trap 以及 Telemetry 协议的数据，可有效提升采集的实时性、准确性和全面性。Telemetry 的引入可支持高频实时数据流传输，有效减少延迟并提升监控效率[43]。同时，智能化协议选择机制可根据网络环境动态调整采集策略[44]，从而提高复杂网络的管理效率。

然而，实际应用中仍面临着诸多挑战，例如协议间数据整合、带宽和延迟优化、实时性与准确性保障，以及系统的可扩展性和低维护成本设计。因此，研究基于多协议融合的异构数据采集方法，不仅是网络管理技术创新的重要方向，也为提升网络运维效率、保障网络环境安全与稳定等基础设施环境运行提供了有效支撑。本章描述的优化方法旨在提升监控精度与实时性，从而进一步提高数据采集的效率与智能化水平，并为智能网络运行状态监控系统设计提供理论与实践依据。

3.2 方法设计

本方法的核心在于融合多种异构协议，利用各协议优势，构建多层次模块化框架[45-46]，从而实现灵活的基础数据采集，进一步提升采集的全面性、准确性和实时性。基于上述方法，整体框架设计了数据采集层、数据融合与处理层、智能调度与协议选择层，整体框架结构见图 3-1。

本方法的设计原则与目标主要是通过融合多种异构协议采集设备的性能指标、连通性、日志等数据，根据场景自适应选取采集协议进行高频数据流采集与低延迟检测，通过高内聚低耦合的模块化架构封装采集协议，方便系统新增采集协议时快速适配。同时，通过协议切换和备份机制[47]，快速应对网络或设备故障，确保采集任务执行的连续性。

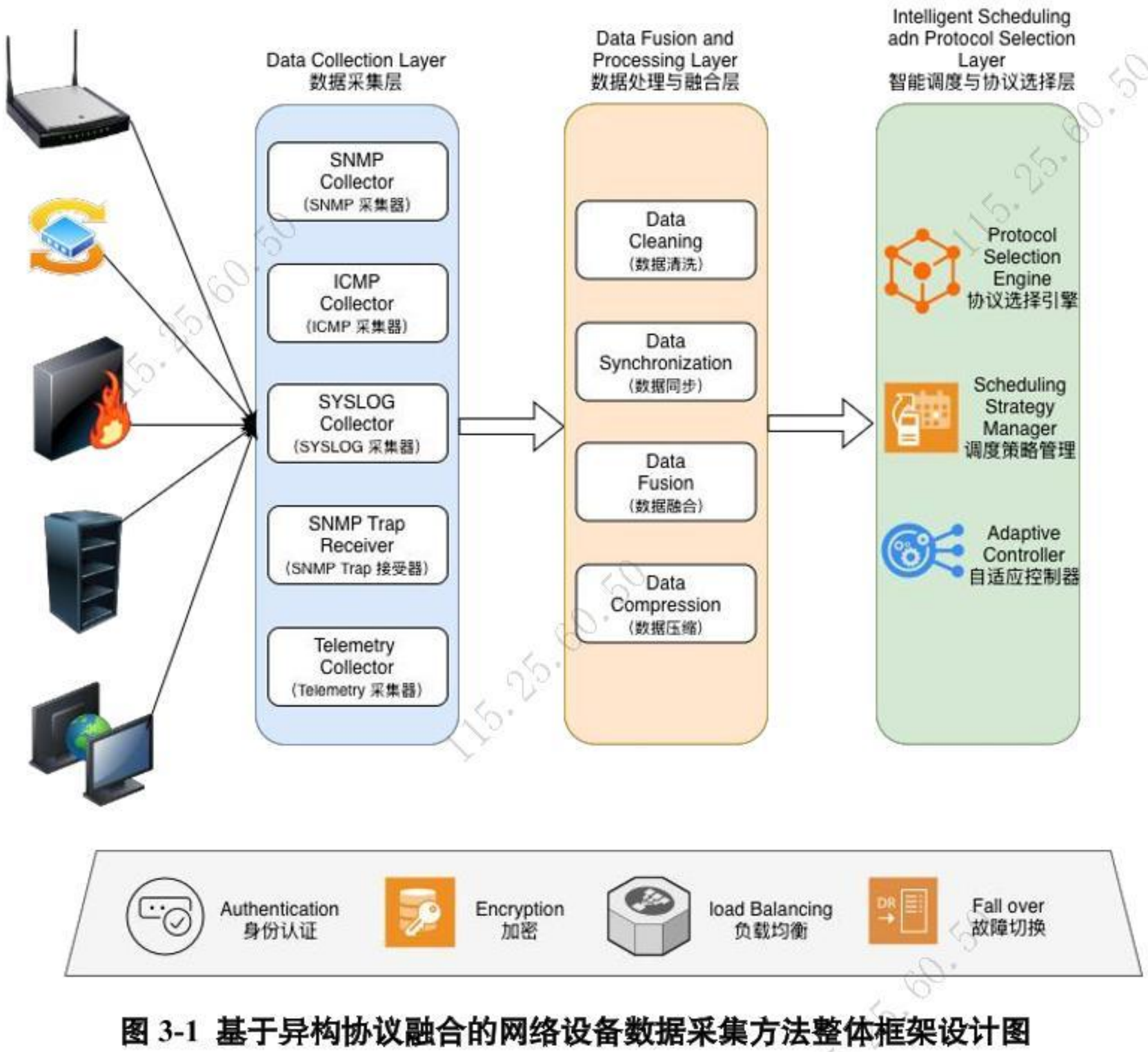


图 3-1 基于异构协议融合的网络设备数据采集方法整体框架设计图

Fig.3-1 Overall Framework Design Diagram of Network Device Data Collection Method Based on Heterogeneous Protocols

数据采集层负责从多种异构的网络设备中获取实时的基础运行数据，通过将 SNMP、ICMP、Syslog、SNMP Trap 及 Telemetry 等协议进行整合选取，构建多源数据输入通道[48]；数据处理与融合层承担数据清洗、格式转换、时间同步和语义融合等任务，主要用于实现多协议数据的统一建模和一致性管理；智能调度与协议选择层则利用规则驱动与智能算法相结合的策略，根据网络运行带宽、设备负载情况、采集延迟及监控精度的需求，动态选择最优协议组合与采集周期。此外，该框架还在设计中引入高可用性与容错机制，当某一协议通道出现异常时，系统可自动切换至备用协议或其他采集路径，以保证监控过程的连续性与稳定性，提高系统的高可用性。上述模块化、可扩展的数据采集方法能够在复杂多变的网络环境下保持高效运转，以满足不同规模与类型网络数据的采集需求。

3.2.1 数据采集层

数据采集层是整个多协议融合框架的基础组成部分，其主要任务是从网络设备中实时、稳定地获取运行状态和性能数据。该层通过整合 SNMP、ICMP、Syslog、SNMP Trap 及 Telemetry 等多种异构协议，构建了一个灵活、高效、可动态调度的数据采集体系，各协议之间既可独立运行，也可协同工作，根据不同设备类型、网络环境和监控需求灵活选择合适的采集方式，从而实现数据的多源同步与补充采集。

该方法中使用的协议及其角色分配见表 3-1。

表 3-1 异构协议及角色分配

Tab.3-1 Heterogeneous Protocols and Role Allocation

协议	角色
SNMP	负责静态监控和定期采集性能数据，如 CPU 使用率、内存使用率、接口流量等[49]，满足非实时数据采集需求
ICMP	负责实时监控网络层的连通性和网络延迟，通过“ping”命令检测链路质量，适用于低延迟、实时性要求较高的场景
Syslog	负责设备的日志采集，记录系统事件和异常信息，为故障排查提供关键信息
Telemetry	负责获取设备的动态数据，提供细粒度的性能指标，适用于高频数据采集和实时监控的场景
SNMP Trap	负责异常时主动向管理系统推送告警信息，适用于故障发生时的即时响应和自动化报警

3.2.2 数据处理与融合层

数据处理与融合层作为多协议融合框架的核心模块，主要负责执行对不同协议采集到的数据进行预处理、清洗、格式化、同步、融合、压缩及优化等任务[50]。该层的核心任务是进行数据清洗、数据同步、数据融合、数据压缩与优化，工作流程见图 3-2。

(1) 数据清洗

数据清洗是数据处理的第一步，旨在剔除无用、冗余和可能的错误数据，确保后续分析过程和存储的数据可靠且有效，清洗过程中主要包含冗余数据剔除、错误数据剔除、缺失数据处理[51]三个方面：

① 冗余数据剔除：不同协议可能会采集相同的数据，特别是在同时使用 SNMP 和 ICMP 时，设备的状态信息可能会重复获取。因此，清洗模块需

要剔除重复采集的数据，避免存储和分析过程的冗余。

② 错误数据剔除：采集过程中可能会出现错误的数据，系统需要通过规则或算法识别并剔除这些无效数据，确保清洗后的数据可用性。

③ 缺失数据处理：部分数据可能由于设备故障、协议问题或网络问题导致数据缺失，针对缺失数据，可以使用插值方法进行补充，以确保数据的完整性。

(2) 数据同步

在多协议数据采集过程中，不同协议采集到的数据可能会存在时间上的差异，因此，数据同步是确保采集数据一致性的关键步骤[52]，该过程主要包括以下两种集中处理方法：

① 统一时间戳：不同协议的数据可能来自不同的时间源，因此在融合数据时，需要通过统一的时间戳机制对所有的数据进行同步，从而使用网络时间协议可以同步所有设备的时间，并为每条数据记录统一的时间标识，用于确保数据分析时的数据一致性。

② 数据时序一致性：将不同协议采集到的数据进行时间序列的对齐，避免因时间偏差导致的分析误差，通过对时间戳进行排序和插值，以确保数据在时间维度上的一致性。

(3) 数据融合

数据融合是该层的核心任务之一，其目的是通过合并来自多个协议的数据，生成一个全面的、统一的数据视图，该方法[53]包括以下三个步骤：

① 协议间数据合并：不同协议采集的数据通常是针对不同维度的。例如，SNMP 协议获取设备性能数据，ICMP 获取链路质量数据，Syslog 获取设备日志信息，系统需要将这些来自不同协议的数据进行合并，以提供设备运行的整体情况。

② 加权融合：为确保数据融合的精确性，根据每种协议的数据质量和时效性，为其赋予不同的权重。例如，Telemetry 可提供高频数据流，数据时效性强，可给予较高的权重，SNMP 数据更新较慢，可适当降低其权重。

③ 数据合并策略：融合过程中，可能会遇到协议间数据冲突的情况，如不同协议采集的设备性能数据不一致，此时需要结合加权平均法、最大值法、最小值法等算法来进行数据合并，以确保数据的准确性。

(4) 数据压缩与优化

数据压缩与优化主要针对大规模的数据采集进行处理，以减少存储的消耗和带宽压力，同时保证数据的准确性与有效性[54]，该方法主要包括以下两

个步骤：

① 数据压缩：针对采集到的高频数据流通过压缩算法进行压缩，以减少存储空间占用和传输带宽消耗。

② 数据优化：采用数据优化算法，将采集到的数据进行整理和归类，可将设备的状态信息和性能数据分开存储、分类存储，方便后续的查询和过程分析。

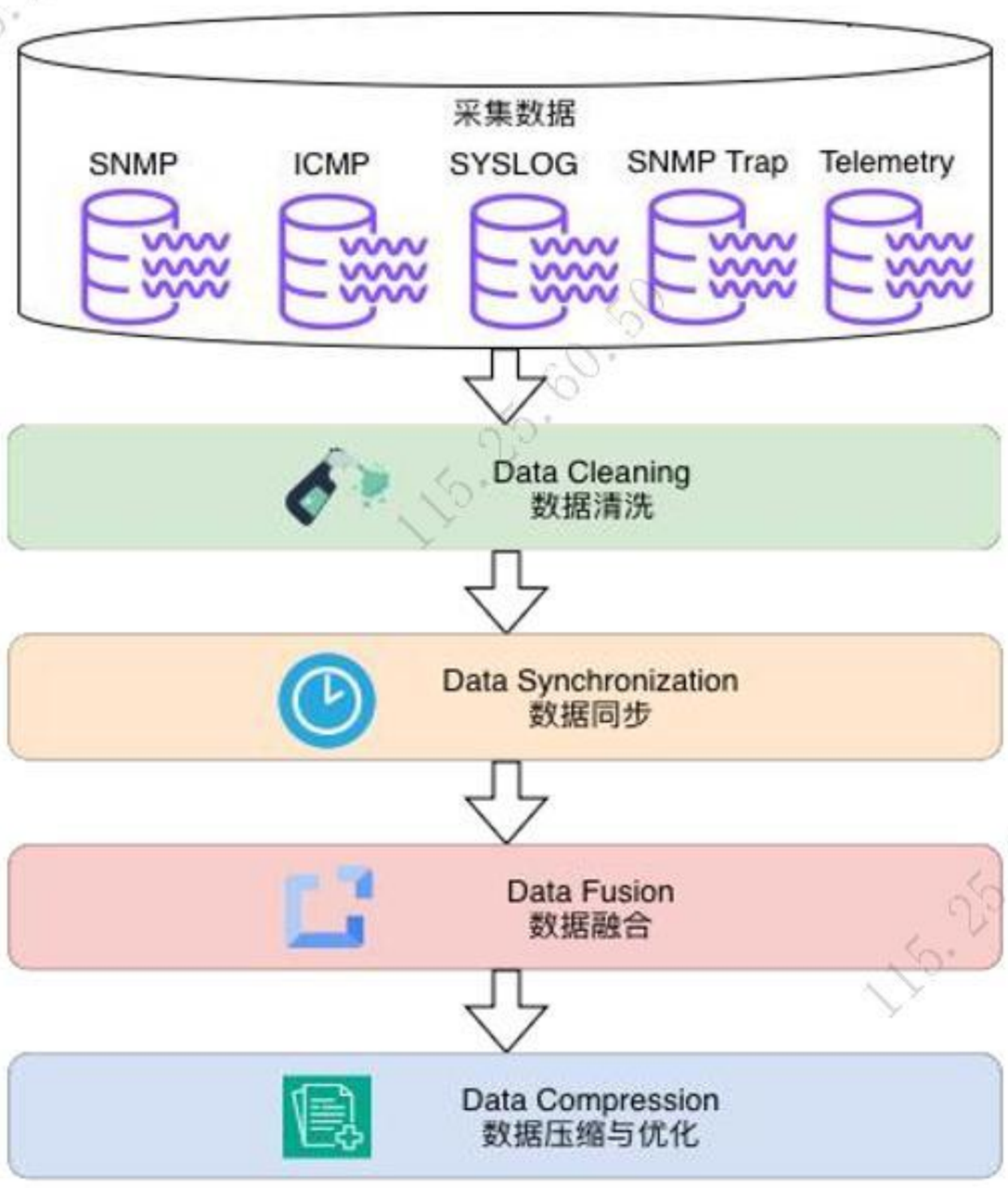


图 3-2 数据融合与处理层核心任务工作流程图

Fig.3-2 Core Tasks Workflow Diagram of Data Fusion and Processing Layer

3.2.3 智能调度与协议选择层

为提高数据采集的效率和响应速度，框架设计了智能调度与协议的选择方法，此方法主要基于以下原则：

(1) 实时性与精度的平衡原则

网络设备的监控过程中，实时性和准确性是两个重要的指标。不同协议具有不同的实时性和精度特征，应根据需要的监控精度和实时性要求来选择合适的采集协议，从而根据网络环境的变化，动态选择最合适的协议进行数据采集。

Telemetry 协议和 ICMP 协议适合实时监控、故障检测等高实时性要求任

务。Telemetry 支持高频次的数据流采集，适用于设备状态变化频繁的场景，如实时性能监控和动态网络环境。ICMP 协议具有较低的延迟和快速响应特性，适用于实时链路监测，当网络负载较低，且仅需监测设备的连通性和链路质量时，选择 ICMP 协议。

SNMP 协议和 Syslog 协议适合设备性能、配置采集等高精度要求任务。SNMP 提供详细的设备统计数据，适用于低频次但高精度的数据采集，如带宽、内存、CPU 使用率等。Syslog 协议通常用于捕获设备运行中的错误和警告，提供高精度的事件日志数据，以便后续的分析或故障排查。

(2) 负载感知调度原则

根据设备和网络的负载情况，智能调度方法可以自动调整采集频率，并选择适当的采集协议，通过监控设备负载、网络带宽和数据流量，选择合适的协议以避免协议过度采集造成带宽瓶颈。

当设备处理能力不足或内存占用过高时，则选择 ICMP 协议进行轻量级的连通性测试，或选择 SNMP Trap 协议获取设备的状态变化，以避免传统的 SNMP 协议轮询带来的负载压力，亦可以通过自动降低 Telemetry 采集频率用于减小负载压力。

当发生网络拥塞或带宽限制导致网络整体带宽不足时，则选择 ICMP 协议进行低带宽消耗的数据采集，或采用 SNMP 协议进行低频轮询，避免带宽的过度占用。如果 Telemetry 协议数据传输量较大时，可通过动态调整其数据包大小或减少采集频率，以减轻带宽压力。

(3) 自动协议切换原则

自动协议切换机制可让系统在不同的网络状态或设备条件下灵活切换采集协议，根据实时数据反馈，当某一协议无法满足采集需求时，可自动切换到备选协议[55]，此机制可帮助确保网络监控系统的高可用性、低延迟性和高效性。

当发生设备不可达或性能下降等设备故障以及异常情况时，可通过 SNMP Trap 协议主动告警或使用 Telemetry 实时监控进行告警和故障检测。如果设备运行效率低于预期，系统应自动切换至 Telemetry 协议进行高频次数据采集，以及时了解设备的故障原因和影响。

当设备恢复正常并且性能稳定时，系统应切换回 SNMP 协议进行低频次的性能监控和状态采集，以减轻系统运行负担，同时确保设备的长期稳定运行。

如果设备的状态良好且网络运行稳定，则系统从高频率的 Telemetry 协

议切换为低频率的 SNMP 协议，从而减少不必要的带宽消耗和计算负载。根据设备状态、网络环境和数据采集需求，表 3-2 给出了协议选择的具体规则集，可用于自动选择协议。

表 3-2 协议选择规则集

Tab.3-2 Protocol Selection Rule Set

网络状态/设备条件	优选协议	备用协议	选择原因
实时性要求高	Telemetry	ICMP	Telemetry 支持高频次实时数据流，ICMP 适用于实时链路监控
精度要求高	SNMP	Syslog	SNMP 提供详细的设备性能数据，Syslog 用于日志分析
设备负载高	SNMP Trap / ICMP	Telemetry	SNMP Trap 可以通过告警方式获取状态，ICMP 消耗小
带宽受限	ICMP	SNMP	ICMP 在低带宽环境下快速响应，SNMP 适用于低频轮询
设备故障	SNMP Trap / Telemetry	SNMP	SNMP Trap 及时获取故障告警，Telemetry 提供高频数据采集
设备恢复	SNMP	Telemetry	SNMP 适用于长期稳定设备的低频监控，Telemetry 适用于高频监控

在智能调度与协议选择层，按照网络环境评估、实时性与精度评估、协议选择、监控与调整四个步骤完成自动协议选择，协议选择原理见图 3-3。

首先，评估当前网络环境，包括带宽使用情况、网络负载、设备状态等。其次，根据当前任务的实时性要求和数据采集精度要求，优先选择合适的协议。随后，结合网络条件和设备状态，动态选择最优的协议进行数据采集。在数据采集过程中，持续监控协议采集效果，如发现网络带宽不足或设备负载过高，将自动调整协议频率或切换协议。

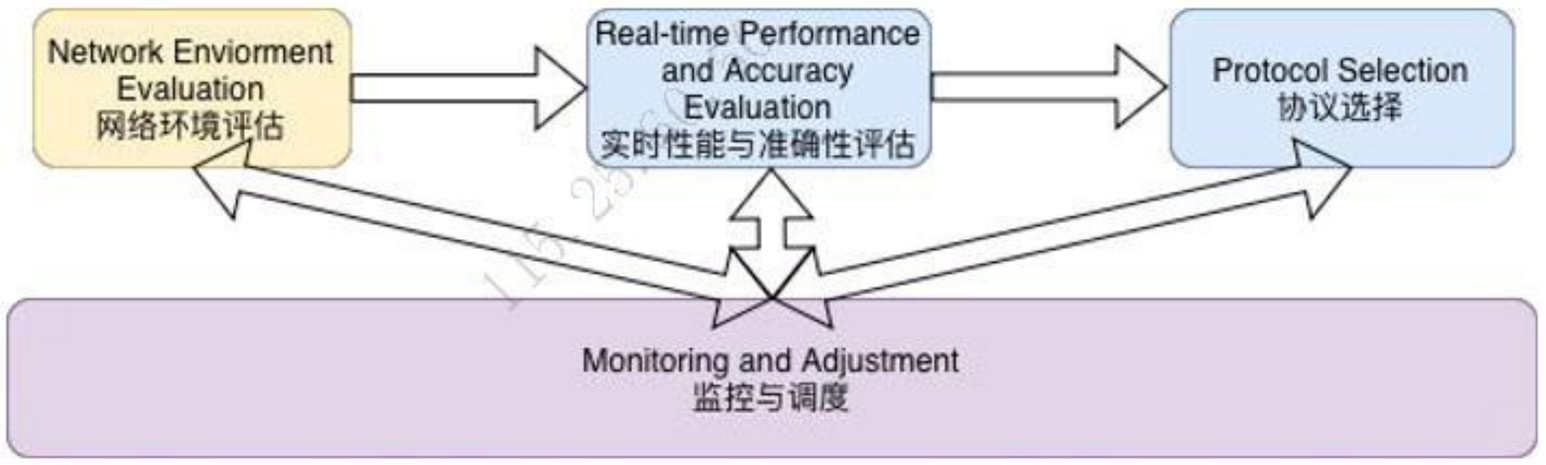


图 3-3 智能调度与协议选择原理图

Fig.3-3 Schematic Diagram of Intelligent Scheduling and Protocol Selection

智能调度与协议选择层的设计使得数据采集系统能够根据实时网络和设备状态灵活调整采集策略。通过实时性与精度的平衡、负载感知调度和自动

协议切换机制，可以在不同的工作环境中优化数据采集，提升网络设备监控的效率和精度[56]。这一机制不仅能提高数据采集的准确性和及时性，还能确保在不同网络条件下，始终保证系统能够稳定运行并保证高可用性。

3.3 实验与分析

为验证本章提出的基于异构协议融合的网络设备数据采集方法的有效性，本章节设置了一组对比实验，将单协议采集方式作为对照方法，从效率、准确性、实时性、灵活性、适应性、容错性与高可用性七个维度对采集方法进行全面的评估。

实验目标为验证异构协议融合采集方法在复杂网络环境中对数据采集效率的提升效果，分析异构协议融合机制在不同网络负载下的适应性与动态调度能力，并对比不同协议间及融合机制下，数据采集的准确性与实用性表现，评估该方法在异常条件下的容错性与高可用性性能。

3.3.1 评价指标

为了全面评估单协议方法和多协议融合方法在数据采集中的表现，本实验设定效率、准确性、实时性、适应性 、容错性和高可用性共 6 个评价指标对实验结果进行量化。下面依次给出评价指标定义。

(1) 效率 (Efficiency)

$$Efficiency=\frac{N_{success}}{N_{total}}\times\frac{1}{T_{avg}} \tag{3-1}$$

其中$N_{success}$为成功采集次数, N_{total}为总采集任务数, T_{avg}为平均采集时间。表示单位时间内成功采集的效率。

(2) 准确性 (Accuracy)

$$StrictAcc=\frac{\sum_{m\in\Omega}(Y_m=\hat{Y}_m)}{|\Omega|} \tag{3-2}$$

Ω 表示所有设备或数据点的集合， Y_m 是设备m的真实值， $\hat{Y}_m$是模型采集的数据值， $|\Omega|$是数据点的总数。

(3) 实时性 (Latency)

$$Latency=\frac{1}{N}\sum_{i=1}^{N}(t_{recv,i}-t_{send,i}) \tag{3-3}$$

其中 $t_{send,i}$、$t_{recv,i}$分别为第 i 次采集请求的发送与接收时间，N 是总的数据请求次数。

(4) 适应性 (Adaptivity Index)

$$AI=\frac{1}{N}\sum_{i=1}^{N}P_i\cdot\ \Delta S_i \tag{3-4}$$

其中 P_i为网络状态变化的概率，ΔS_i为切换后采集效率提升的比例，反映系统对网络状态变化的自适应能力。

(5) 容错性 (Fault Tolerance)

$$FT=1-\frac{E_{fail}}{E_{total}} \tag{3-5}$$

E_{fail}是采集失败事件数，E_{total}是总的采集事件数。

(6) 可用性 (Availability)

$$System\ Availability\ Rate=\frac{T_{available}}{T_{total}} \tag{3-6}$$

$T_{available}$是系统可用的总时间，T_{total}是总的实验时间。

3.3.2 实验环境及数据集

实验平台包括 12 台虚拟化网络设备，其中模拟核心交换机、汇聚路由器、边缘设备各 4 台，配备 Intel Xeon 2.6GHz CPU、32GB 内存、1Gbps 链路速率，所有设备均部署了 SNMP Agent、Syslog Server、Telemetry Exporter、Trap Receiver 及 ICMP 回显功能。

系统运行在 Ubuntu 22.04 环境下，数据采集方法则使用了 Python 和 Go 语言进行编写。

采样周期设定为 10 秒，实验持续 30 分钟，重复 10 轮，最终取平均值。系统分别运行 SNMP、Syslog、Telemetry 等单协议采集模式和本章节提出的异构协议融合的多协议采集模式。

3.3.3 对比方法

本实验对照方法为单独使用 SNMP、ICMP、Syslog、Telemetry、SNMP Trap 协议进行网络设备数据采集的情况。

3.3.4 实验方案

异构协议融合采集中，系统将根据网络状态、设备负载与数据类型动态选择最优采集协议，因此智能调度机制包含以下三个原则：

(1) 实时性与精度平衡机制

若采集周期较长或网络延迟较高，则系统优先选择 Telemetry 协议通过流式传输保证实时性；若采集数据为设备状态类或关键性能指标，则系统优先采用 SNMP 协议以提高采集精度。

(2) 负载感知调度机制

当链路利用率 $>80\%$ 或设备负载较高时，则系统自动降低 SNMP 协议的轮询频率，切换至低负载的 Telemetry 和 ICMP 路径；当日志流量剧增时，则触发异常检测模块，以减少 Telemetry 更新率。

(3) 自动协议切换机制

当主协议响应失败次数≥3 时自动触发切换，优先切换顺序为：SNMP → Telemetry → ICMP → Syslog → SNMP Trap → SNMP，循环往复。

对应的实验方案如表 3-3 所示。

表 3-3 异构协议融合采集实验方案

Tab.3-3 Experimental Scheme for Heterogeneous Protocol Fusion Acquisition

网络状态	主协议	辅助协议	切换条件	调度目标
正常	SNMP	Telemetry	RTT < 200 ms	精度优先
高负载	Telemetry	ICMP	丢包率 > 5%	减负载
日志异常	Syslog	SNMP Trap	异常日志检测	快速告警
设备异常	SNMP Trap	ICMP	心跳失败 ≥3 次	快速恢复
链路不稳定	ICMP	SNMP	RTT 波动 > 100 ms	稳定性优先

3.3.5 实验结果及分析

本实验中，分别使用了 5 种单协议方法和异构协议融合方法，在不同网

络负载、带宽受限的环境下进行数据采集，并根据实验结果对数据采集效率、数据准确性、数据实时性、容错性与高可用性进行统计和分析。

实验结果如表 3-4 所示。

表 3-4 单协议与异构协议融合采集方法性能对比

Tab.3-4 Performance Comparison of Single-Protocol and Heterogeneous Protocol Fusion Acquisition Methods

协议类型	效率 Efficiency (%)	准确性 Accuracy (%)	实时性 Latency (ms)	适应性 Adaptability Index (score)	容错性 Fault Tolerance (score)	可用性 Availability (%)
SNMP	86.3	91.8	310	80	70	92.5
ICMP	79.4	84.1	210	72	60	89.3
Syslog	83.1	88.5	260	77	68	91.0
SNMP Trap	88.2	90.4	240	84	75	93.1
Telemetry	92.5	94.6	150	90	82	95.6
单协议平均值	85.9	89.9	234	80.6	71	92.3
异构协议融合	**95.7**	**97.8**	**120**	**94**	**89**	**97.2**

通过对比单协议方法和异构协议融合方法，实验结果表明：

采集效率上，Telemetry 得益于其流式采集机制在单协议采集中效率最高，达到了 92.5%；ICMP 采集效率最低，主要因其采集内容有限且频繁探测受限；多协议融合方法通过并行采集与动态调度，综合效率显著提升至 95.7%。

准确性上，单协议采集过程中，SNMP 与 Telemetry 具备较高的准确性；ICMP 与 Syslog 准确性较低，主要由于其受网络抖动和日志延迟影响较大；融合方法可利用多源冗余验证机制，将准确性提高到 97.8%。

实时性上，单协议延迟平均约 234 ms，而融合架构可通过智能调度将延迟优化至 120 ms，通过自动切换协议保持稳定的时延。

适应性上，SNMP Trap 与 Telemetry 在单协议中表现较为突出，而多协议融合方法通过策略动态配置，可将适应性提升至 94 分。

容错性与高可用性上，单协议在链路丢失或设备异常时易产生中断，融合方法通过多通道冗余与动态切换，可以保持 97.2%的高可用性和 89 分的容错性，远高于单协议均值。

各协议性能指标对比雷达图见图 3-4。

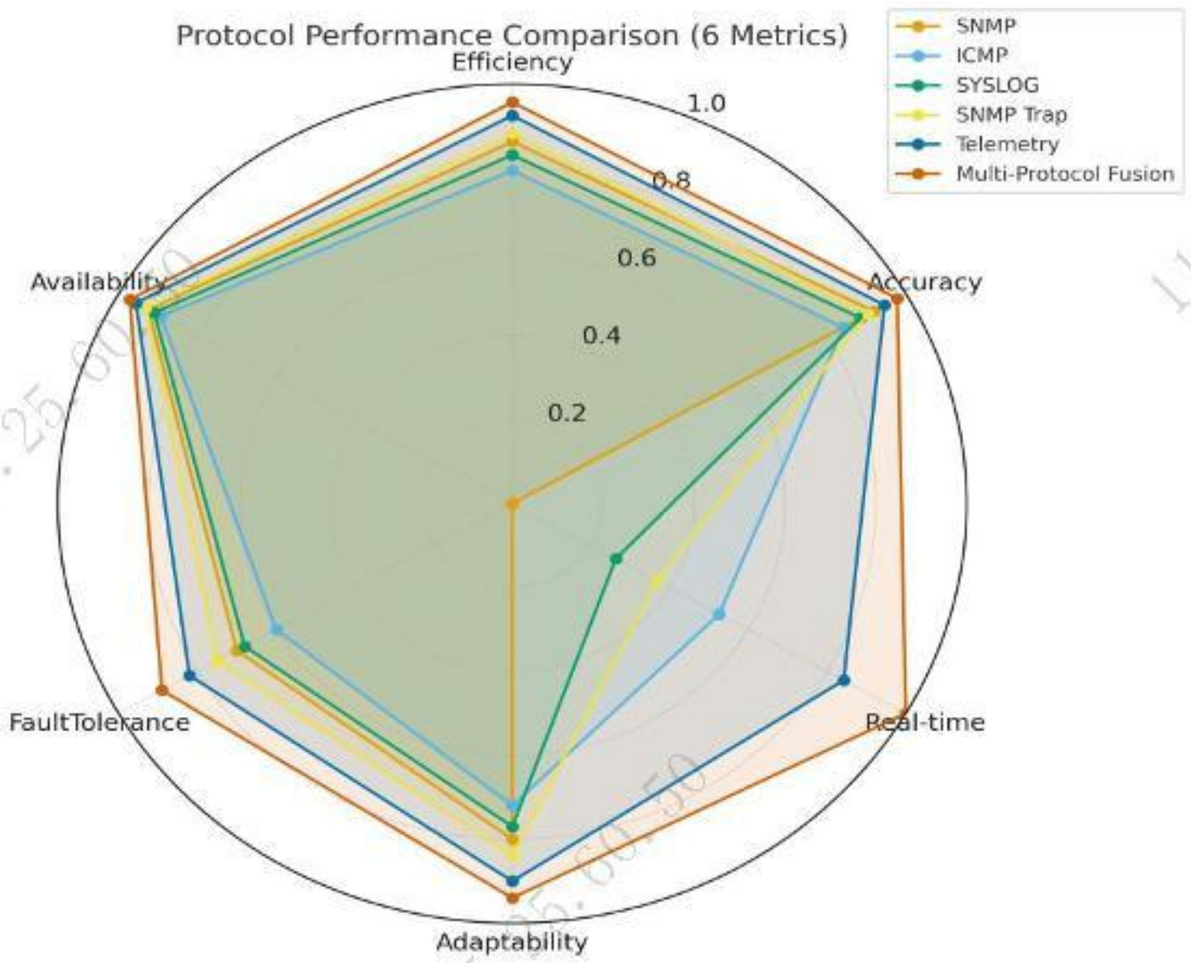


图 3-4 各协议性能指标对比雷达图

Fig.3-4 Protocol Performance Radar Chart

基于单协议的方法在数据采集效率、实时性、准确性等方面存在明显局限，特别是在网络负载高或带宽受限的情况下，单一采集协议的适应性较差，容易导致数据丢失或采集延迟的情况出现。异构协议融合方法则通过动态选择的方法，选择当前场景下最合适的采集协议，使其在高负载、低带宽、设备故障等环境下能够仍然提供稳定、高效、准确的数据采集能力，通过协议间的数据融合，不仅提高了数据的准确性，还提升了容错能力和高可用性。

3.4 本章小结

本章围绕基于异构协议融合的网络设备数据采集方法展开研究，针对传统单协议采集方式在实时性、兼容性与可扩展性方面的不足，提出了一种模块化的多协议融合框架，该框架集成了 SNMP、ICMP、Syslog、SNMP Trap 及 Telemetry 等多种网络管理与监控协议，充分利用了不同协议在数据类型、采集机制及实时性能方面的互补优势，实现了多源数据的协同采集与融合处理。

整体设计上，框架采用分层结构，主要包含数据采集层、数据处理与融合层以及智能调度与协议选择层。数据采集层用于实现多协议并行采集，以确保信息来源的全面性与数据获取的实时性；数据处理与融合层通过数据清

洗、时间同步与语义融合等技术，建立一套统一的数据结构，为后续的数据分析与决策提供高质量的数据支撑；智能调度与协议选择层基于规则与智能算法的结合，对网络状态、设备负载和带宽占用等因素进行动态评估，实时调整协议的使用优先级和采集周期，从而在系统性能与资源消耗之间寻找一个最优的平衡点。

方法充分考虑了协议协同、数据融合、高可用性与容错性等关键问题。当某一协议出现异常或设备通信中断时，可自动切换至备用协议或其他采集通道，确保数据采集任务的连续性与稳定性。同时，通过模块化设计，可以使框架具备良好的可扩展性与兼容性，能够方便地集成新协议或扩展新的数据采集模块，以适应未来网络环境和技术发展的需要。

实验结果表明，该方法在采集效率、数据准确性、实时响应能力、系统灵活性以及容错性能等方面均优于传统单协议采集模式。多协议融合框架不仅显著提升了网络设备监控系统的整体性能，也为后续基于智能分析与自适应告警机制的监控系统研究提供数据基础和技术支撑。

综上所述，本章提出的基于异构协议融合的网络设备采集方法有效解决了大规模复杂网络环境下的数据采集瓶颈问题，实现了网络设备监控的高效化、智能化与可持续性，为后续系统实现与性能优化奠定了坚实的理论与实践基础。

4 基于知识推理的多算法融合自适应动态基线阈值算法

4.1 引言

在网络设备运行状态监控中，CPU 利用率、内存占用率、接口流量、丢包率、错误报文数等指标通常呈现复杂的时序特征。由于网络拓扑、协议类型及业务负载动态变化，这些指标具有非线性、非平稳和周期波动性[57]的特点。传统的静态阈值方法无法适应这种变化，导致阈值固化、误报率高、漏检频繁等问题。3σ法等动态基线方法虽能部分适应波动，但仍存在非正态分布或多峰数据中易产生误差、缺乏语义理解能力、依赖方差等数值特征调整权重、权重优化过程缺少反馈校正与全局最优保证等关键问题。

因此，如何在异构多协议网络环境中构建一种具备语义理解与自学习能力的动态基线阈值算法[58-59]，使其在不同设备、指标与场景下实现准确、稳定、低误报的状态监控成为网络设备智能监控的重要研究方向。

针对上述问题，我们提出一种基于知识推理驱动的多算法融合自适应动态基线阈值算法，该算法在传统多算法融合动态基线框架的基础上，引入大模型[60]作为知识推理引擎，通过理解设备上下文语义与运行状态特征，动态调整各算法权重，并通过规则推理引擎对大模型结果施加逻辑约束，实现语义推理和规则约束的双层知识驱动机制，使模型在面对复杂网络动态特性时，既具备大模型的知识泛化能力，又保持了规则逻辑的一致性和收敛性，从而确保算法的正确性与工程的可控性。

4.2 任务定义

4.2.1 问题描述

设网络设备在监控周期[1,T]内采集的指标数据序列为：

$$D=\{x_1,x_2,...,x_T\},x_t \in R$$

本章任务的目标是构建动态基线函数$f_t(\theta)$，使得在任意时刻 t ，可自适应地计算出上下阈值：

$$B_t^{up}\ ,B_t^{low}=f_t(D_{t-w:t}\ ,\theta_t) \tag{4-1}$$

其中，θ_t表示时间 t 的算法权重参数集，w 为时间窗口大小。若当前观测值 $x_t \notin [B_t^{low}, B_t^{up}]$，则判定该时刻存在异常。

自适应动态基线阈值方法的核心任务即为：通过引入知识推理[61]机制，在多算法融合框架下实现动态权重 $\theta_t=(\alpha_t,\beta_t,\gamma_t)$ 的自适应优化[62-63]，使基线阈值动态反映系统运行状态变化。

4.2.2 数学建模

(1) 多算法融合基线模型

采用三类典型算法生成独立基线$B_t^{3\sigma}, B_t^{smooth}, B_t^{Comp}$分别对应传统统计型基线[64-66]、趋势平滑基线[67]与补偿修正基线[68]。

融合基线计算为：

$$B_t=\alpha_t B_t^{3\sigma}+\beta_t B_t^{smooth}+\gamma_t B_t^{Comp} \tag{4-2}$$

约束条件为：

$$\alpha_t+\beta_t+\gamma_t=1 \ , \alpha_t , \beta_t , \gamma_t \geq 0$$

(2) 知识推理驱动的权重动态优化模型

设系统的目标损失函数为：

$$L(W_t)=\lambda_1 \cdot FAR(W_t)+\lambda_2 \cdot (1-Precision(W_t)) \tag{4-3}$$

其中，FAR 为误报率，$Precision$ 为精确率。通过最小化损失函数可得到最优权重向量：

$$W_t^*=arg\min_{W_t} L(W_t) \tag{4-4}$$

然而直接优化需要大量样本与计算代价，因此，通过引入大模型知识推理模块，借助语义分析输出候选权重调整方案：

$$W_t^{cand}=\Phi_{LLM}(F_t,C_t,H_t) \tag{4-5}$$

其中，W_t^{cand}表示由大模型推理得到的权重向量，F_t代表方差、波动率、趋势因子等实时统计特征信息，C_t代表协议类型、设备类别、网络层级等设备上下文信息，H_t代表异常类型、误报率、阈值漂移趋势等历史监控知识，Φ_{LLM} 为知识推理映射函数。

根据领域知识和逻辑规则对候选权重进行约束修正：

$$W_t^{LLM'}=\Psi_{Rule}(W_t^{cand},R) \tag{4-6}$$

其中 R 为规则集，Ψ_{Rule}为逻辑推理函数。

最终组合输出：

$$W_t=(1-\lambda)W_t^{cand}+\lambda W_t^{LLM'} \tag{4-7}$$

其中λ为规则约束强度系数。

(3) 动态阈值生成与修正机制

为平衡统计数据与语义推理结果，引入指数加权更新机制，进行权重平滑与收敛：

$$W_t=(1-\eta)W_t^{old}+\eta W_t^{LLM} \tag{4-8}$$

其中$\eta\in(0,1)$表示知识融合系数。

最终的动态阈值定义为：

$$B_t^{up}=B_t+k\sigma_t \ ,\ B_t^{low}=B_t-k\sigma_t \tag{4-9}$$

若监控系统在当前窗口内的误报率$FAR_t>F_{th}$，则执行阈值放宽：

$$B_t^{up,new}=B_t^{up,old}+\delta\sigma_t \ ,\ B_t^{low,new}=B_t^{low,old}-\delta\sigma_t \tag{4-10}$$

4.3 算法设计

4.3.1 算法总体设计

本章提出的算法旨在通过多算法协同建模与自适应加权机制，动态拟合网络设备运行状态并自动调整阈值上下限。该方法的核心思路包括三方面：

(1) 多维数据建模

融合多协议采集的网络运行指标，实现多源数据的统一建模。

(2) 多算法基线融合

采用统计型（3σ）、平滑型（Smooth）与补数修正型（Complement）三种基线算法进行加权融合。

(3) 自适应阈值调整

引入动态权重机制，通过知识模型推理[69]动态调整融合权重，实现阈值的自学习和实时优化。

方法整体框架设计如图 4-1 所示。

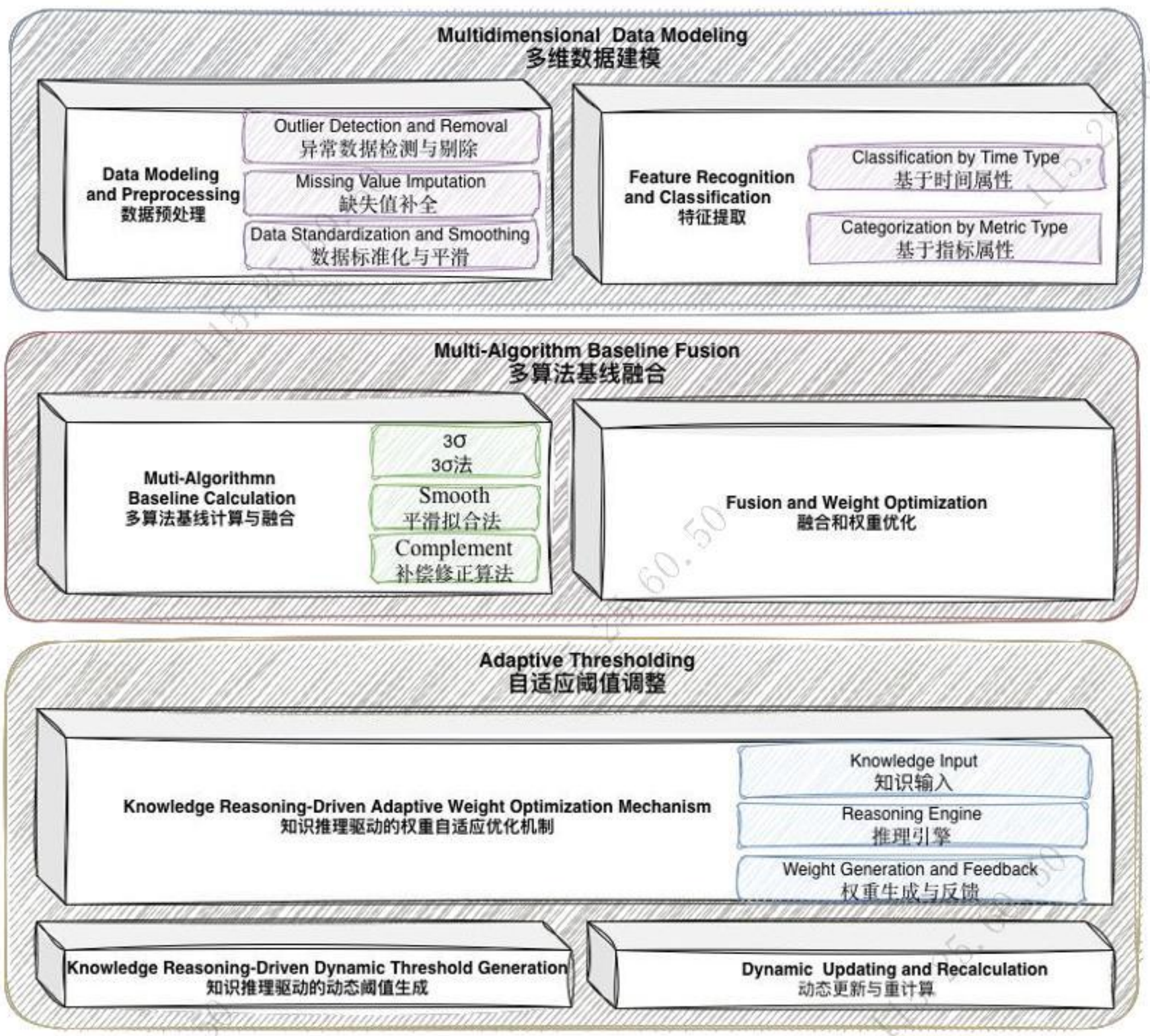


图 4-1 基于知识推理的多算法融合自适应动态基线阈值方法整体框架设计图

Fig.4-1 Overall Framework Design Diagram of Knowledge-Reasoning Multi-Algorithm Fusion Adaptive Dynamic Baseline Thresholding

算法包括数据预处理[70]、特征提取、多算法基线计算、融合与权重优化、知识推理驱动的权重自适应优化机制、知识推理驱动的动态阈值生成、动态更新与重计算几个核心步骤。首先将多协议监控采集到的异构数据转换为统一时序格式，对异常点和缺失值进行预处理，根据时间及指标属性进行特征识别与分类，结合概率统计方法、平滑拟合算法及补偿机制，利用加权融合策略，将多算法输出形成动态基线，根据实时特征输入、上下文知识库及历史数据，基于大模型知识推理和规则约束生成针对当前运行状态的最优权重组合，实现在线自适应优化。具体算法逻辑见算法 4.1。

算法 4.1 KR-MFA-DBT: Knowledge Reasoning–Driven Multi-Algorithm Fusion Adaptive Dynamic Baseline Threshold Method

Input: Monitoring data $D = \{x_1, x_2, \ldots, x_T\}$; window size w; learning rate η; adjustment factor δ; rule set R; parameter k.

Output: Adaptive thresholds $\{B_t^{up}, B_t^{low}\}_{t=w}^{T}$.

Initialize weights $W_0 = [\alpha_0, \beta_0, \gamma_0]$.

Compute initial baselines $B_0^{3\sigma}$, B_0^{smooth}, B_0^{comp}, $B_0 \leftarrow \alpha_0 B_0^{3\sigma} + \beta_0 B_0^{smooth} + \gamma_0 B_0^{comp}$.

for $t = w$ **to** T **do**

Extract feature vector $F_t = [\mu_t, \sigma_t, v_t, r_t]$.

Retrieve contextual and historical knowledge C_t, H_t.

Compute base baselines $B_t^{3\sigma}$, B_t^{smooth}, B_t^{comp}.

Fuse baselines:

$$B_t = \alpha_t B_t^{3\sigma} + \beta_t B_t^{smooth} + \gamma_t B_t^{comp}.$$

Infer candidate weights using LLM:

$$W_t^{LLM} = \Phi_{LLM}(F_t, C_t, H_t).$$

Apply rule reasoning to refine weights:

$$W_t^{rule} = \Psi_{Rule}(W_t^{LLM}, R).$$

Update weights:

$$W_t = (1 - \eta) W_{t-1} + \eta W_t^{rule}.$$

Compute thresholds:

$$B_t^{up} = B_t + k\sigma_t, \quad B_t^{low} = B_t - k\sigma_t.$$

if $FAR_t > F_{th}$ **then** adjust thresholds:

$$B_t^{up} = B_t^{up} + \delta\sigma_t, \quad B_t^{low} = B_t^{low} - \delta\sigma_t.$$

end if

end for

return $\{B_t^{up}, B_t^{low}\}_{t=w}^{T}$.

(1) 数据预处理

动态基线计算的精度高度依赖于输入数据质量，预处理包括异常数据检测与剔除、缺失值补全、数据标准化与平滑三个核心步骤。

为确保基线计算的稳定性，对通过 SNMP、ICMP、Syslog、Telemetry

等多协议源融合采集的指标数据序列 $D=\{x_1,x_2,...,x_T\}$进行以下预处理：

① 异常数据检测与剔除

采用基于标准差的异常值检测规则，对于样本数据集 ，若满足：

$$|x_t-\mu|>2.5\sigma \tag{4-11}$$

则该点被视为异常并剔除。该规则能有效去除突发异常对后续基线计算的影响。

② 缺失值补全

若时间序列中存在缺失值，则通过补数算法补全，补数算法如下所示：

$$B_i^{up}=max\left(B_{基准}^{up},\frac{B_{i-1}^{up}+B_{i+1}^{up}}{2}\right) \tag{4-12}$$

$$B_i^{low}=min\left(B_{基准}^{low},\frac{B_{i-1}^{low}+B_{i+1}^{low}}{2}\right) \tag{4-13}$$

此算法不仅平滑过渡，还可防止局部异常导致的上下阈值失衡。

③ 数据标准化与平滑

为减少短期波动对基线拟合的影响，先对指标序列进行归一化处理：

$$x'_t=\frac{x_t-min(x)}{max(x)-min(x)} \tag{4-14}$$

随后采用五点或七点三次平滑公式进行噪声消减。

该模块保证后续基线估计不受偶发波动干扰，为多算法融合奠定数据质量基础。

(2) 特征提取

对时序数据提取关键特征：

$$F_t=[\mu_t,\sigma_t,\nu_t,r_t] \tag{4-15}$$

其中，μ_t为滑动均值，σ_t为波动标准差，ν_t为波动率，r_t为趋势斜率。

结合上下文信息C_t、历史监控知识H_t，形成特征语义联合输入。

(3) 多算法基线计算与融合

多算法融合是本文的核心思想。本模块将多种算法优势进行互补性集成，形成复合型基线计算体系。

① 3σ法

假设指标数据服从近似正态分布 $N(\mu,\sigma^2)$，则动态阈值可表示为：

$$\begin{cases} B_{upper}=\mu+3\sigma \\ B_{lower}=\mu-3\sigma \end{cases} \tag{4-16}$$

其中

$$\mu=\frac{1}{n}\sum_{i=1}^{n}x_i,\quad \sigma=\sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i-\mu)^2}$$

该算法具有计算简单、解释性强的特点，能快速响应数值波动，但在噪声较高或趋势变化显著时稳定性不足，非正态数据中容易出现误差。

② 平滑拟合法（Smooth）

为提高对周期性波动指标的稳定拟合，引入五点三次平滑算法：

$$Y_i'=\frac{1}{35}(-3Y_{i-2}+12Y_{i-1}+17Y_i+12Y_{i+1}-3Y_{i+2}) \tag{4-17}$$

该方法能有效消除短期随机扰动，使基线更贴合长期趋势，提高基线的连续性与趋势保真度。

③ 补偿修正算法（Complement）

针对采样异常、丢包或指标跳变造成 3 σ 和拟合结果不连续或存在空缺点位时，采用邻域均值补偿策略进行补数修正，从而保证上下界连续：

$$B_i^{修正}=\frac{B_{i-1}+B_{i+1}}{2} \tag{4-18}$$

修正后的基线经过再平滑处理，可确保曲线平滑且无突变，该方法在网络拓扑变动或数据缺失时能有效修复局部异常。

为平衡不同算法的优缺点，充分发挥三种算法的互补性，基于式 (4-2) 的定义实现多算法融合的权重调整。不同算法在并行计算后输出基线结果，形成具有多层鲁棒性的综合基线。

(1) 知识推理驱动的权重自适应优化机制

知识推理驱动的权重自适应优化机制是本研究的核心，运行逻辑如图 4-2 所示，由知识输入模块、推理引擎、权重生成与反馈三个部分构成。在知识输入部分接收实时特征数据、上下文信息和历史知识三种输入，在推理引擎部分通过规则推理与语义推理结合生成最优权重决策，在权重生成与反馈部分将推理得到的权重经平滑处理。实现从数据驱动向知识驱动的动态权重优化转变。

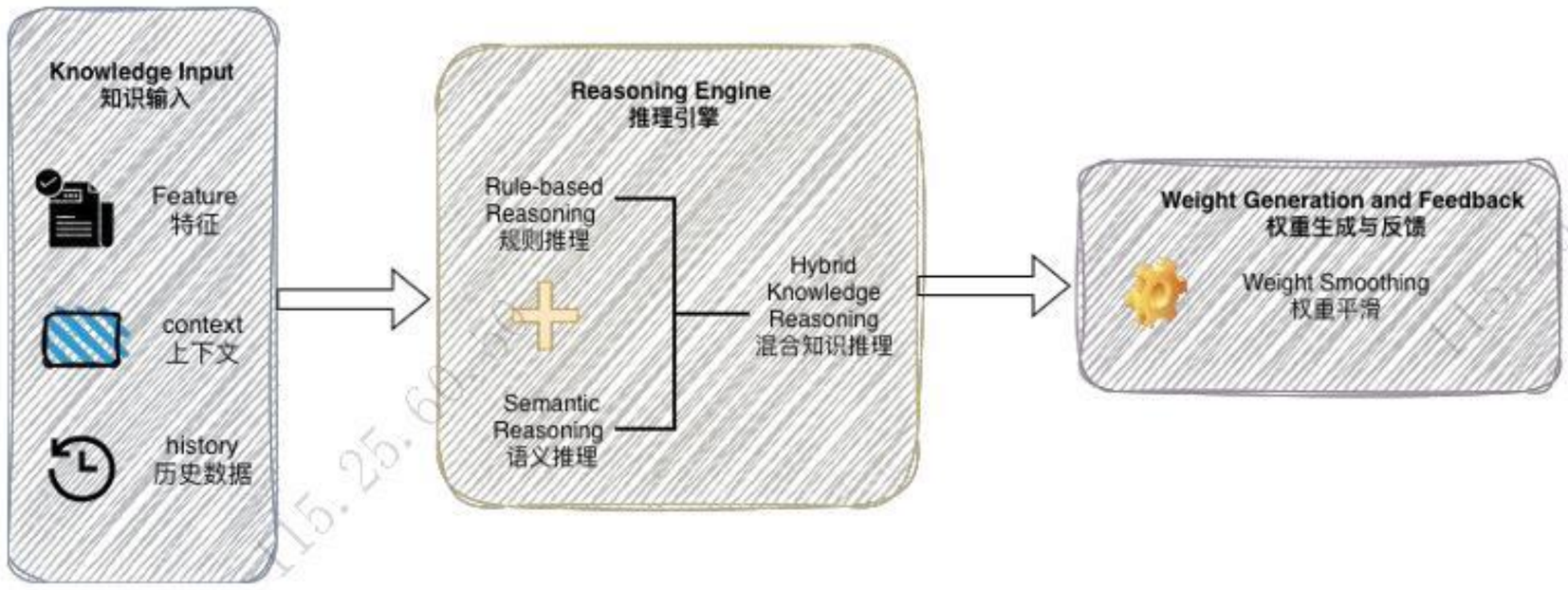


图 4-2 知识推理驱动的权重自适应优化机制运行逻辑

Fig.4-2 Operational Logic of the Knowledge Reasoning-Driven Weight Adaptive Optimization Mechanism

大模型根据F_t,C_t,H_t组合特征执行语义推理，输出权重调整结果W_t^{LLM}，根据领域知识和逻辑规则对候选权重进行约束修正，从而得到权重结果$W_t^{LLM'}$，并根据式（4-7）、式（4-8）进行权重平滑与收敛最终得到W_t。推理结果不仅可提供数值权重，还可生成自然语言解释，如检测到协议 Telemetry 波动频率较高时，可增加平滑算法权重。

其中，规则推理机制基于领域专家经验与统计规律预定义规则库，对不同波动状态和指标类型采用不同的权重调整策略，规则形式化描述如下：

$$\begin{cases} \alpha_t\uparrow & \text{若 } V_t<0.1 \text{ 且指标稳定（如 CPU、内存）} \\ \beta_t\uparrow & \text{若 } 0.1\le V_t\le 0.3 \text{ 且为周期性流量指标} \\ \gamma_t\uparrow & \text{若 数据缺失率>5\% 或存在突发尖刺} \end{cases}$$

(2) 知识推理驱动的动态阈值生成

采用式（4-9）生成最终动态上下基线。当系统检测到阈值偏离或误报率上升时，根据式（4-10）自动触发放宽或收紧操作，保证检测灵敏度与稳定性。

(3) 动态更新与重计算机制

当流量模式或架构变更导致历史数据不再代表当前规律时，系统触发基线重计算机制，该机制流程见图 4-3。通过周期性重计算，保证基线时效性与鲁棒性。

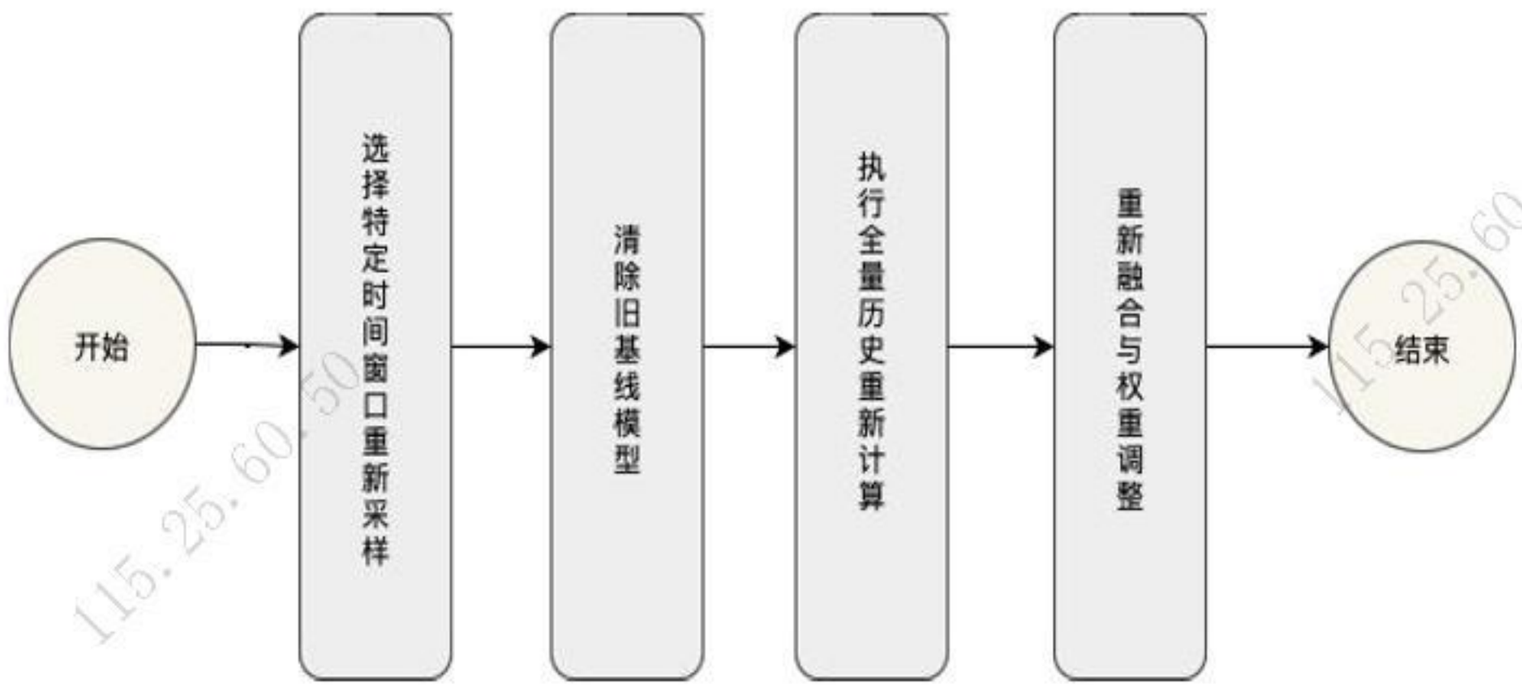


图 4-3 动态更新与重计算机制流程图

Fig.4-3 Flowchart of the Dynamic Update and Recalculation Mechanism

4.3.2 创新性体现

本研究的创新点主要体现在将大模型引入动态基线阈值算法中，通过语义推理动态调整融合权重，并通过规则推理引擎对候选结果施加逻辑约束，实现对不同设备、协议及运行模式的智能适配。结合 3σ 法、平滑算法、补偿算法的数值分析优势与 LLM 的语义推理优势，形成数值语义双驱动机制。同时，提出基于凸组合的权重更新形式，确保算法在有限步内进行收敛，同时可输出推理理由，从而具备可解释性。

4.3.3 算法正确性与收敛性证明

根据权重更新公式：

$$W_{t+1}=(1-\eta)W_t+\eta\Phi_{LLM}(F_t,C_t,H_t) \tag{4-19}$$

因为Φ_{LLM}的输出满足归一化约束且收敛于有限空间Δ_3，更新过程为凸组合，满足：

$$\|W_{t+1}-W_t\|\le(1-\eta)\|W_t-W_{t-1}\| \tag{4-20}$$

由 Banach 不动点定理可知，序列$\{W_t\}$为 Cauchy 序列，当$t\to\infty$时存在极限W^*，算法收敛于稳定解。

4.3.4 时间复杂度分析

表 4-1 给出本算法各个模块的主要计算及时间复杂度，分析可得综合复

杂度为：

$$T(n)=O(n)$$

该算法保持线性复杂度，适合实时监控场景。

表 4-1 算法时间复杂度分析

Tab.4-1 Analysis of Algorithm Time Complexity

模块	主要计算	时间复杂度
数据预处理	滑动窗口统计	$O(n)$
多算法基线计算	3σ + 平滑 + 补数	$O(n)$
大模型推理	外部 API 调用	$O(1)$
权重融合更新	加权计算	$O(n)$

4.4 实验与分析

本节旨在通过实验验证本章提出的基于知识推理驱动的权重自适应优化机制在网络设备运行状态监控场景中的性能表现。

通过对比传统静态阈值方法、基于波动率调整的多算法融合方法与本文提出的知识推理优化方法，从检测准确性、实时性、误报率、稳定性与解释性五个维度进行综合评估。实验的目标包括验证大模型推理机制对多算法权重动态调整的有效性、探讨语义推理在多协议异构网络数据中的适应性、分析知识推理机制对误报抑制和检测稳定性的贡献、验证大模型知识推理的解释性与跨设备泛化能力。

4.4.1 评价标准

为了客观评估本章提出方法的性能，实验综合采用精确率、召回率、F1 值以及误告警率四项指标进行量化评估。

(1) 精确率（Precision）

$$Precision=\frac{TP}{TP+FP} \tag{4-21}$$

(2) 召回率（Recall）

$$Recall=\frac{TP}{TP+FN} \tag{4-22}$$

(3) F1 值（F1-Score）

$$F1=\frac{2\times Precision\times Recall}{Precision+Recall} \tag{4-23}$$

(4) 误报率（FAR）

$$FAR=\frac{FP}{FP+TN} \tag{4-24}$$

4.4.2 实验环境与数据集

实验环境配置见表 4-2。

表 4-2 实验环境与配置说明

Tab.4-2 Description of Experimental Environment and Configuration

环境类型	参数配置
服务器配置	Intel Xeon Silver 4310 × 2, 64GB RAM, Ubuntu 22.04
数据库	Oracle
模型引擎	Python 3.11 + PyTorch 2.0
大模型	GPT- 4 API
协议数据源	SNMP、ICMP、Syslog、Telemetry
实验时长	连续监控 7 天，共 8640 个采样点

在该实验中，大模型 GPT-4 被用于执行语义推理任务，包括设备语义理解、异常上下文判断与权重调整策略生成，推理结果以 JSON 格式传回权重优化模块，实现可解释、可复现的参数动态调整。

实验数据来自企业级网络数据中心的实际运行日志与监控数据，数据覆盖核心路由器、汇聚交换机和接入交换机三类设备类型，Telemetry、SNMP、Syslog 和 ICMP 四种常见采集协议，接口吞吐率、转发表变化率、CPU 利用率、内存占用率、丢包率、延迟六项网络运行指标，共采集 17418240 条数据，随机采样 5%形成包含 870912 条数据的实验数据集。

4.4.3 对比算法

为全面验证本章提出的基于大模型知识推理驱动的权重自适应优化机制的有效性与优势，本研究设计了三种典型的对比方法：

(1) 静态阈值法（Static）

静态阈值法采用人工经验设定的固定阈值，对各类指标统一设定上下限。当数据超出该范围时触发告警。

(2) 多算法动态融合法（MFA-DBT）

基于统计特征进行权重调整，将 3 σ 法、平滑法和补偿法进行加权融合。

(3) 本文方法（KR-MFA-DBT）

在多算法动态融合的基础上引入大模型知识推理机制，通过融合规则推理与语义推理动态调整权重，兼顾数值精度与语义可解释性，实现认知级阈值生成。

4.4.4 实验结果与分析

4.4.4.1 实验结果

为验证自适应动态基线阈值方法在真实网络监控中的表现，本研究设计了一套多阶段实验方案。

从实际网络运行环境中收集 Telemetry、SNMP、ICMP、Syslog 四类协议的指标数据，通过时间同步、缺失插补、异常值剔除与 Z-score 标准化，构建统一的时序数据集，使用 Static、MFA-DBT、KR-MFA-DBT 三种方法分别生成上下阈值序列。对于 KR-MFA-DBT，模型在每个时间窗口内调用大模型 GPT-4 进行知识推理，以生成自适应权重组合。

对每个时刻的数据点进行判断，若超过上下基线则标记为异常，作为实验预测值；人工标注的真实异常事件作为真实值，计算 Precision、Recall、F1、FAR 和 Latency 等指标。同时，对 KR-MFA-DBT 输出的推理日志进行审阅，分析其在不同场景下的解释一致性和逻辑合理度。实验在连续 7 天监控数据上进行，最终结果如表 4-3 所示。

表 4-3 三种算法总体性能比较

Tab.4-3 Overall Performance Comparison of the Three Methods

方法	Precision	Recall	F1	FAR
Static	0.791	0.664	0.721	0.105
MFA-DBT	0.858	0.767	0.810	0.061
KR-MFA-DBT	0.932	0.881	0.905	0.033

从结果中可以看出，KR-MFA-DBT 在 Precision 与 Recall 方面均显著高于对比模型，FAR 较 MFA-DBT 下降约 45.9%。这说明大模型在动态权重调整中的语义推理作用显著增强了模型的自适应性与认知合理性。

图 4-4 展示了 KR-MFA-DBT 在接口流量监控中的检测结果。

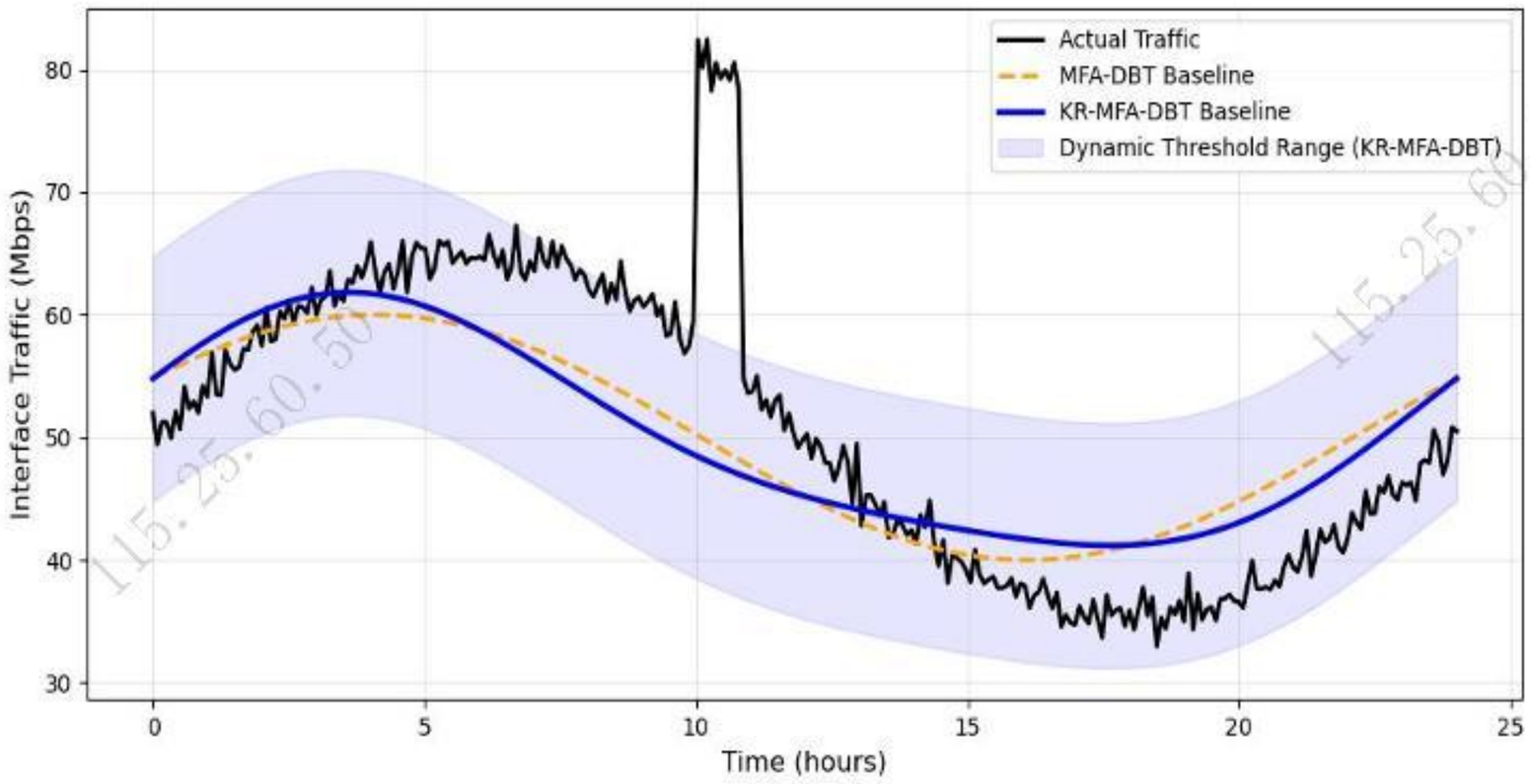


图 4-4 KR-MFA-DBT 算法在接口流量指标上的检测结果曲线

Fig.4-4 Detection Results of the KR-MFA-DBT Method on Interface Traffic Metrics

与传统 MFA-DBT 相比，本研究算法生成的基线曲线能更精确地随趋势波动而自适应调整。当流量出现短时突升或周期性波动时，模型能够通过大模型语义推理动态调整平滑权重β_t，有效减少误报并保持检测敏感度。

可见，大模型知识推理使得系统在面对复杂波动模式时能够更好地理解指标语义，而非仅凭数值变化进行判断，这正是本文方法的核心优势。

4.4.4.2 收敛性实验分析

为验证 KR-MFA-DBT 算法在动态基线权重优化过程中是否能够稳定收敛，对比 Static、MFA-DBT、KR-MFA-DBT 三种算法在多周期训练中的参数变化趋势，证明本文算法在权重迭代中能够逐步趋于稳定。

以权重变化率和基线波动率作为评估指标：

$$\Delta W_t = \| W_t - W_{t-1} \|_2 \tag{4-25}$$

$$\Delta B_t = \frac{|B_t - B_{t-1}|}{B_t} \tag{4-26}$$

收敛条件定义为$\Delta W_t < \varepsilon_\omega, \Delta B_t < \varepsilon_b$，其中$\varepsilon_\omega = 10^{-3}$，$\varepsilon_b = 10^{-3}$。

实验采用滚动窗口方式持续更新基线，窗口长度$\omega=60$。每轮迭代均记录权重向量$W_t=[\alpha_t,\beta_t,\gamma_t]$的变化曲线，并统计 30 个周期的平均收敛速度。

表 4-4 展示了三种算法的收敛对比结果：

表 4-4 各算法收敛结果对比

Tab.4-4 Comparison of Convergence Results of Each Method

方法	平均收敛周期（轮）	平均权重波动率（ΔW）	平均基线波动率（ΔB）	是否收敛
Static	不适用	0.000	0.082	否
MFA-DBT	10	0.007	0.030	中等
KR-MFA-DBT	8	0.004	0.019	强

Static 方法不进行权重迭代，因此收敛周期不适用；其基线波动主要由固定阈值与数据波动引起。MFA-DBT 多算法融合，但仅用波动率等统计规则自适应权重收敛速度中等，体现了融合算法的优势，但仍弱于引入知识推理的 KR-MFA-DBT。KR-MFA-DBT 的权重与基线波动均更小，收敛速度更快，说明知识推理能稳定并加速权重收敛。

图 4-5 展示三种方法随训练轮数ΔW_t的下降趋势。

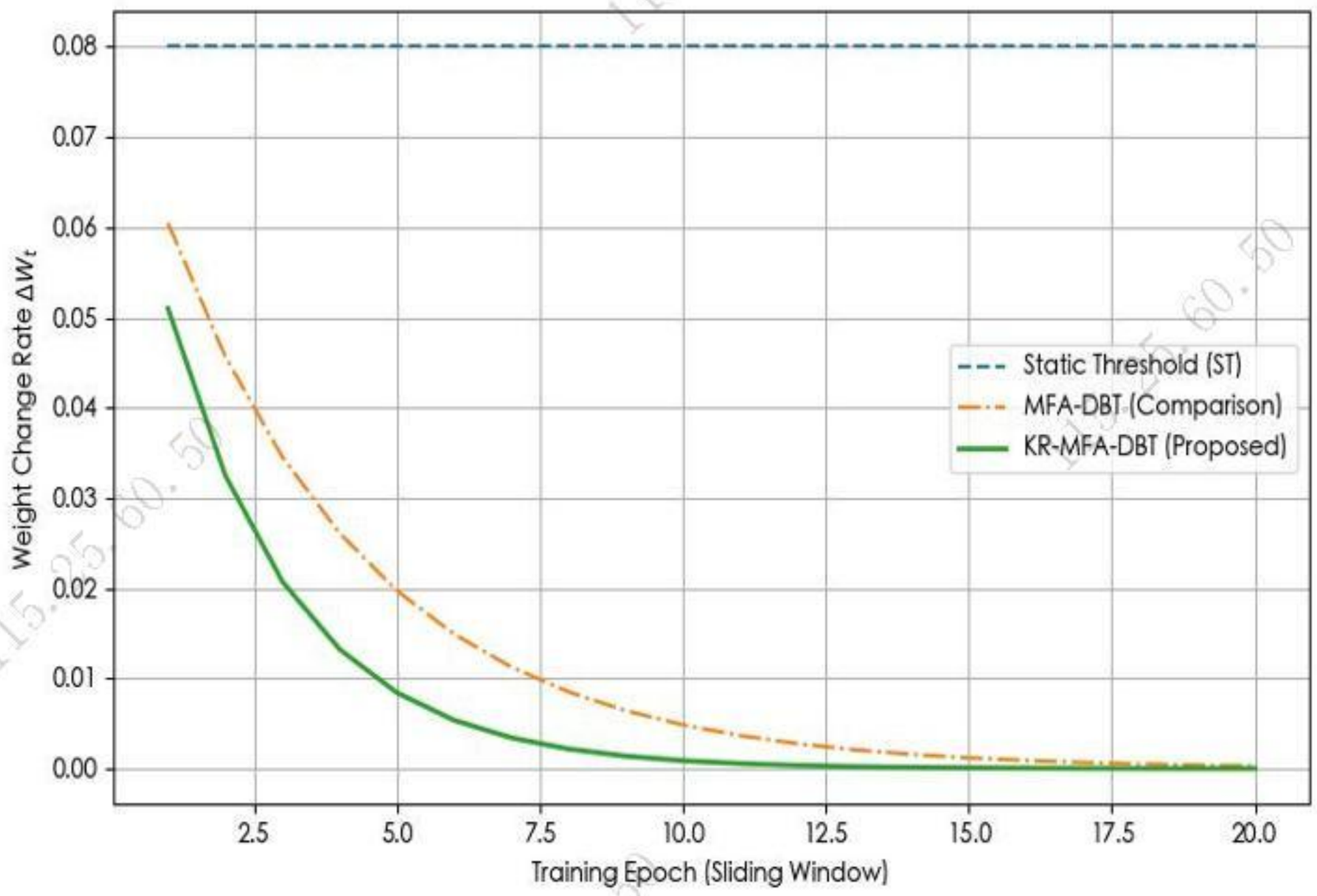


图 4-5 权重变化率收敛曲线

Fig.4-5 Convergence Curve of Weight Update

4.4.4.3 推理过程分析

为进一步验证 KR-MFA-DBT 的可解释性与决策逻辑合理性，选取了典型推理日志进行分析。图 4-6 为大模型输出的推理示例。

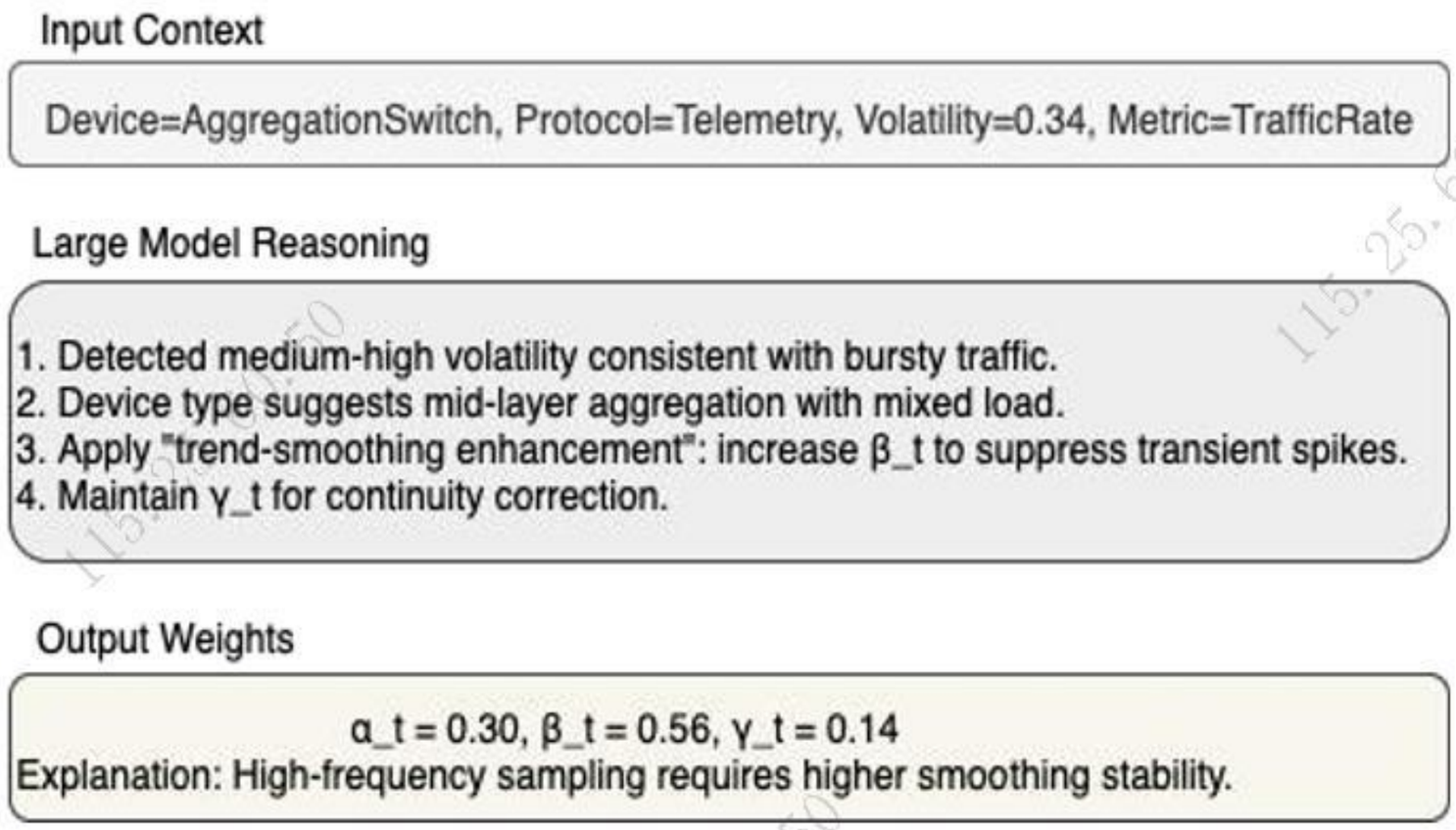


图 4-6 KR-MFA-DBT 方法的推理示例

Fig.4-6 Reasoning Example of the KR-MFA-DBT Method

该日志说明大模型不仅执行了权重调整，还给出了逻辑推断路径和自然语言解释。通过日志可清晰了解调整原因，从而增强了模型的透明度与可验证性。

4.4.4.4 实验结论

通过与静态阈值法和传统多算法融合法的系统对比，本章节提出的KR-MFA-DBT 在精度、稳定性、实时性和智能性方面均实现显著突破，在F1 指标上较不使用知识推理的 MFA-DBT 提升 9.5%，FAR 下降 45%。同时，通过实验证明了该方法的收敛性。KR-MFA-DBT 方法从数值驱动进化为知识驱动，创新性地使智能网络运维阈值建模由统计阶段向认知阶段转变，为构建高自治、高可解释的智能监控系统提供了可行路径。

4.5 本章小结

本章围绕网络设备运行状态监控中阈值动态调整与异常检测的关键问题，针对传统方法存在的阈值固化、灵敏度不足、误报率高与缺乏语义理解能力等不足，提出了一种基于大模型知识推理驱动的多算法融合自适应动态基线阈值方法（KR-MFA-DBT）。该方法从网络设备运行状态监控的实际需求出发，实现了面向复杂网络运行环境的动态阈值自适应调整与异常检测。

通过在真实网络数据集上的对比实验结果可以发现，KR-MFA-DBT 在F1 指标上较传统的多算法融合算法 F1 提升约 9.5%、误报率降低约 45%，显

著优于不具备知识推理的模型，充分验证了该方法在精度、稳定性、实时性方面的优势。此外，该方法还能输出基于大模型生成的推理日志，解释参数调整原因与异常识别逻辑，显著提升了系统的可理解性与可维护性。

综上，该算法有效提升了动态阈值模型的性能，为构建高自治、高可靠性的智能网络设备监控系统提供了新的理论支持与技术路径，实现了网络监控系统从阈值设定自动化向阈值理解智能化的跨越。

5 网络设备运行状态智能监控系统实现与测试

5.1 引言

本章在系统总体设计的基础上，对网络设备运行状态智能监控系统的技术架构、功能模块实现以及系统测试与验证过程进行了详细阐述。

5.2 系统技术架构

本系统的技术架构采用多层分布式设计，整体结构自下而上包括服务器层、操作系统层、数据库层、中间件层、开发框架层和前端框架层。各层之间通过标准化接口和消息中间件实现解耦协作，形成了一个高性能、高可靠性、易扩展的运行体系。系统技术架构如图 5-1 所示。

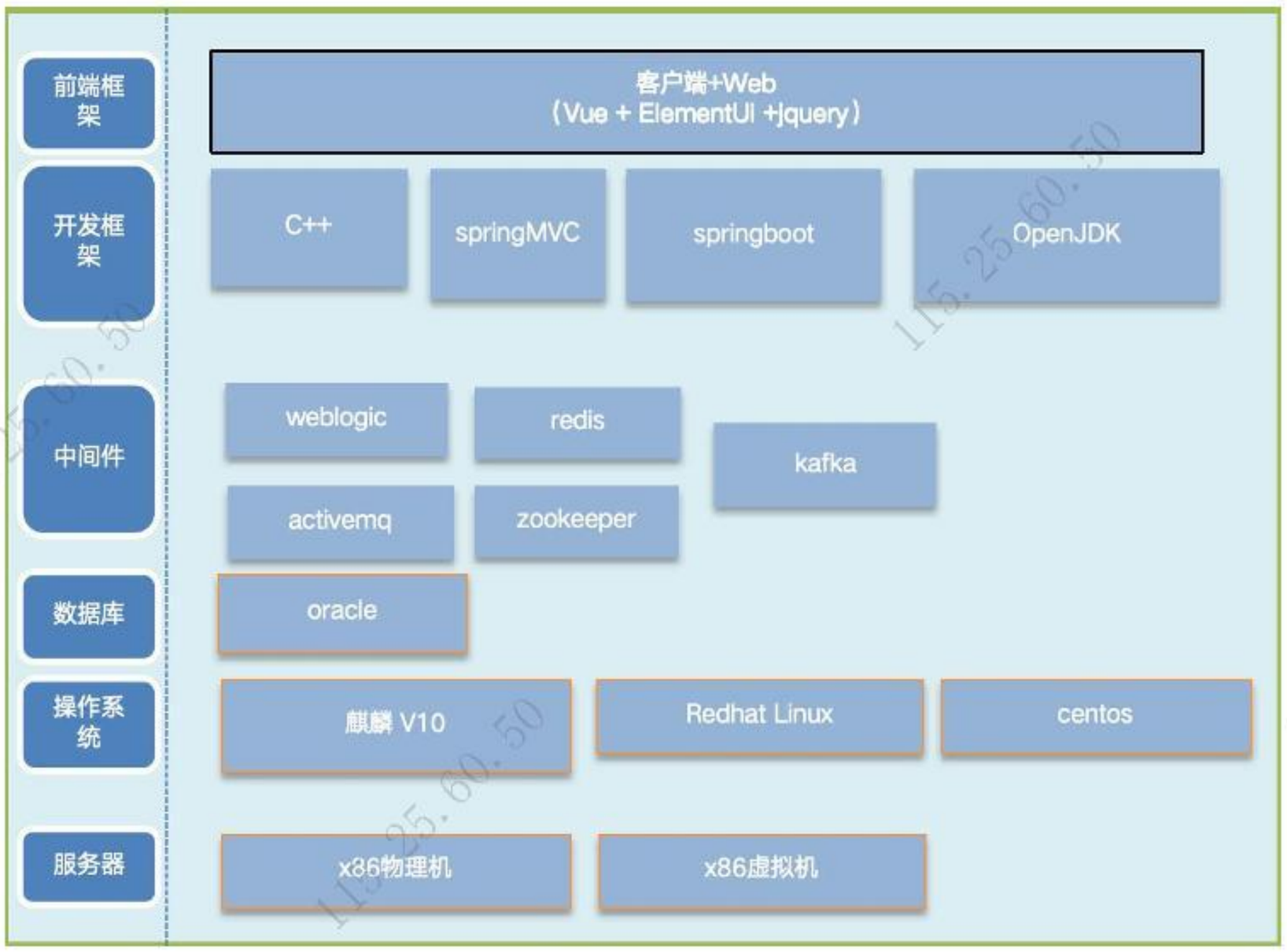


图 5-1 系统技术架构图

Fig.5-1 System Technical Architecture Diagram

在服务器层，系统运行基于 x86 架构的物理机与虚拟化节点，物理服务器主要承载核心数据库和业务逻辑模块，虚拟化节点用于分布式部署采集任务与缓存服务，从而实现资源的灵活调度与弹性扩展。

操作系统层支持麒麟 V10、RedHat Linux 与 CentOS 等多种 Linux 环境，

结合系统日志审计与防火墙策略，为上层服务提供稳定、安全的运行支撑。

数据管理方面，系统采用 Oracle 数据库作为核心数据存储平台，用于存放设备信息、监控配置、性能指标及日志数据，配合多维索引、分区表与主从同步机制，确保数据访问的高效性与可靠性。

中间件层在系统中承担关键的通信协调与异步任务调度功能，其中 WebLogic 负责应用服务的运行与事务管理, Redis 用于状态缓存与会话维护, Kafka 与 ActiveMQ 共同实现采集数据流、任务调度与告警事件的消息传递, Zookeeper 负责服务注册与分布式一致性管理。

在开发框架层, 系统采用 SpringMVC 与 SpringBoot 作为主要的服务开发框架，以模块化微服务架构实现业务逻辑分离与独立部署。C++语言模块负责高性能计算任务，如网络数据采集与实时计算，通过 JNI 与 Java 模块交互以提升处理效率。OpenJDK 作为系统运行环境, 保证跨平台兼容性与稳定性。

前端部分基于 Vue.js、Element UI 与 jQuery 构建，实现了组件化、响应式的交互界面与高效的数据可视化功能。

前后端通过 RESTful 接口通信，结合 ECharts 图表引擎实现实时数据展示与多维趋势分析，满足用户对直观监控和高交互性的需求。

整体上，该技术架构考虑了分层设计、性能优化与模块化扩展等方面的设计需求，保证了系统的安全性与稳定性，为后续系统扩展和性能提升提供技术基础。

5.3 系统功能模块实现

系统包括监控展示、设备管理、任务管理、Ping 采集、聚合规则管理、智能告警、日志管理和性能报表八个功能模块，各模块基于松耦合设计，通过消息中间件实现异步通信，确保系统在高并发场景下的稳定性与扩展性。

5.3.1 监控展示模块实现

监控展示模块是系统的可视化中枢，承担设备运行状态、告警信息、趋势数据等内容的展示与交互控制功能，能够支持视图钻取，支持地域级、厂商级、设备级和告警级多维视图展示，实现层次化监控与告警趋势的追踪。该模块面向运维人员，提供图形化监控界面与实时可视化操作，其功能组成如下：

(1) 网络状态总览

展示全网设备的在线、离线、告警状态等运行状态，支持视图钻取功能，提供厂商、设备、告警三级视图钻取能力。用户可从总览层逐步下钻至设备实例层，以便从宏观到微观地分析告警来源与设备异常模式。

(2) 性能监控面板

实时展示 CPU、内存、带宽、接口流量、连接数等核心指标，可自定义指标显示内容及刷新频率。

(3) 趋势分析视图

基于时序数据库的指标变化趋势展示，支持历史对比与缩放回溯，可按设备、指标、时间段进行多维分析。当用户点击趋势图中的峰值节点时，系统自动联动表格定位至对应告警设备，实现趋势图、表格、告警详情多层级联动分析。

(4) 阈值与告警总览

通过折线图展示告警动态基线阈值的变化趋势，实现历史阈值与实时阈值的统一呈现，通过汇总 Kafka 流式数据和数据库中的历史数据信息，使用基于知识推理的多算法融合自适应动态基线阈值调整方法，实时调整多算法在基线阈值中的系数，动态更新基线阈值。

本模块的前端采用 Vue.js + Element UI + jQuery + ECharts 构建动态仪表盘与趋势图，实时数据更新通过 WebSocket 长连接保证毫秒级响应，提供统一的 RESTful API 接口支持数据查询和交互控制。

图 5-2 展示了区域视图的数据库字段设计，可记录告警的数量、统计区间、网络设备数、端口数、设备类型等字段信息。

GPP_WEBM_PM_REGION_COUNT 的列

区域视图-设备端口统计数据表

	名称	类型	可为空	默认	注释
1	ID	VARCHAR2(32)			告警统计数据id
2	STAT_TIME	NUMBER(13)	Y		数据统计时间
3	REGION	VARCHAR2(128)	Y		区域
4	DEVICE_CNT	NUMBER(19)	Y		总台数
5	PORT_CNT	NUMBER(19)	Y		总端口数
6	SIM1	VARCHAR2(64)	Y		预留字段1
7	SIM2	VARCHAR2(64)	Y		预留字段2
8	SIM3	VARCHAR2(64)	Y		预留字段3
9	DEVICE_TYPE	VARCHAR2(128)	Y		设备类型

图 5-2 区域视图的数据库字段设计

Fig.5-2 Database Field Design for Regional View

图 5-3 展示了网格视图中当前网络设备主、次要告警数据、端口数、风扇异常数、内存和 CPU 的使用率以及设备温度。搜索栏设置了时间筛选、设备类型筛选、搜索按钮以及实时刷新按钮，数据默认每分钟刷新一次。当部分设备或厂商上报数据字段不全时，系统自动识别缺失项并以默认值填充，

避免因数据缺损导致的展示中断。此外，采用索引优化与异步加载机制，显著提升大规模设备列表的查询性能。

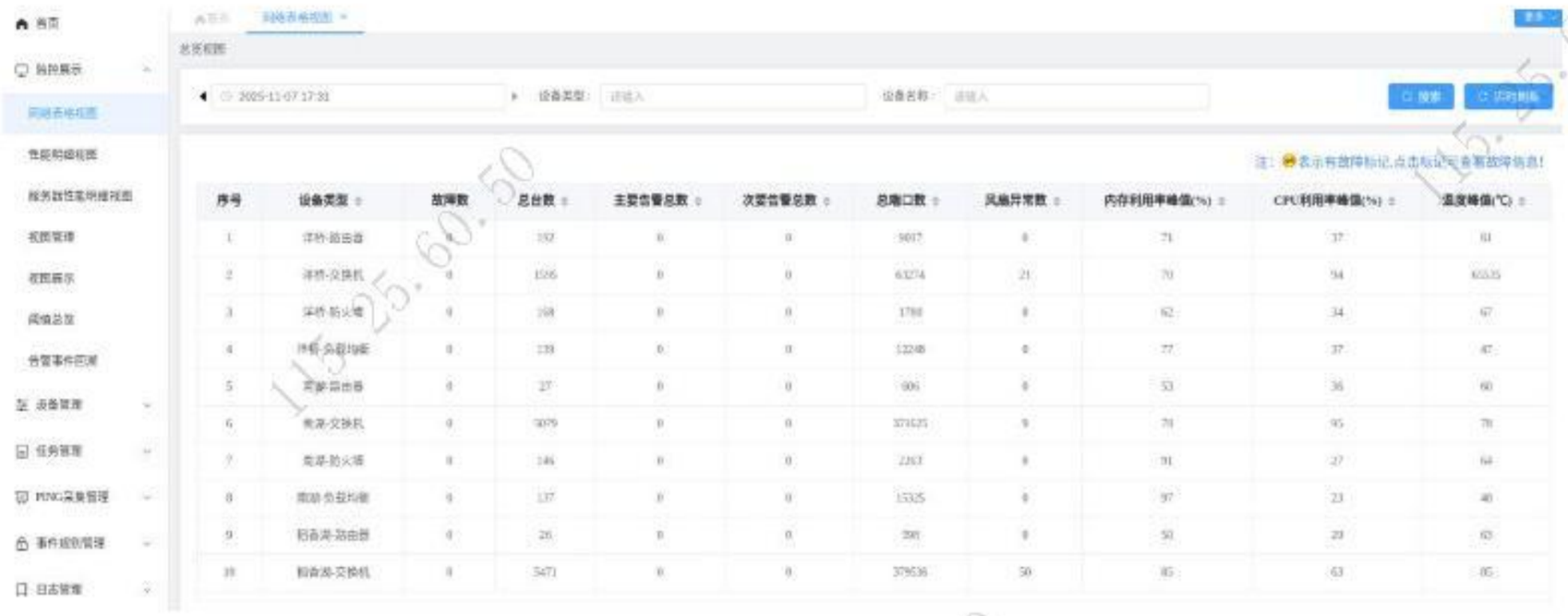


图 5-3 网格视图监控展示

Fig.5-3 Grid View Monitoring Display

5.3.2 设备管理模块实现

设备管理模块位于系统应用服务层的基础层位置，负责所有被监控设备的注册、配置、分组和监控属性维护，通过标准化接口与数据采集、任务管理及可视化展示实现数据交互，构建完整的设备生命周期管理体系。

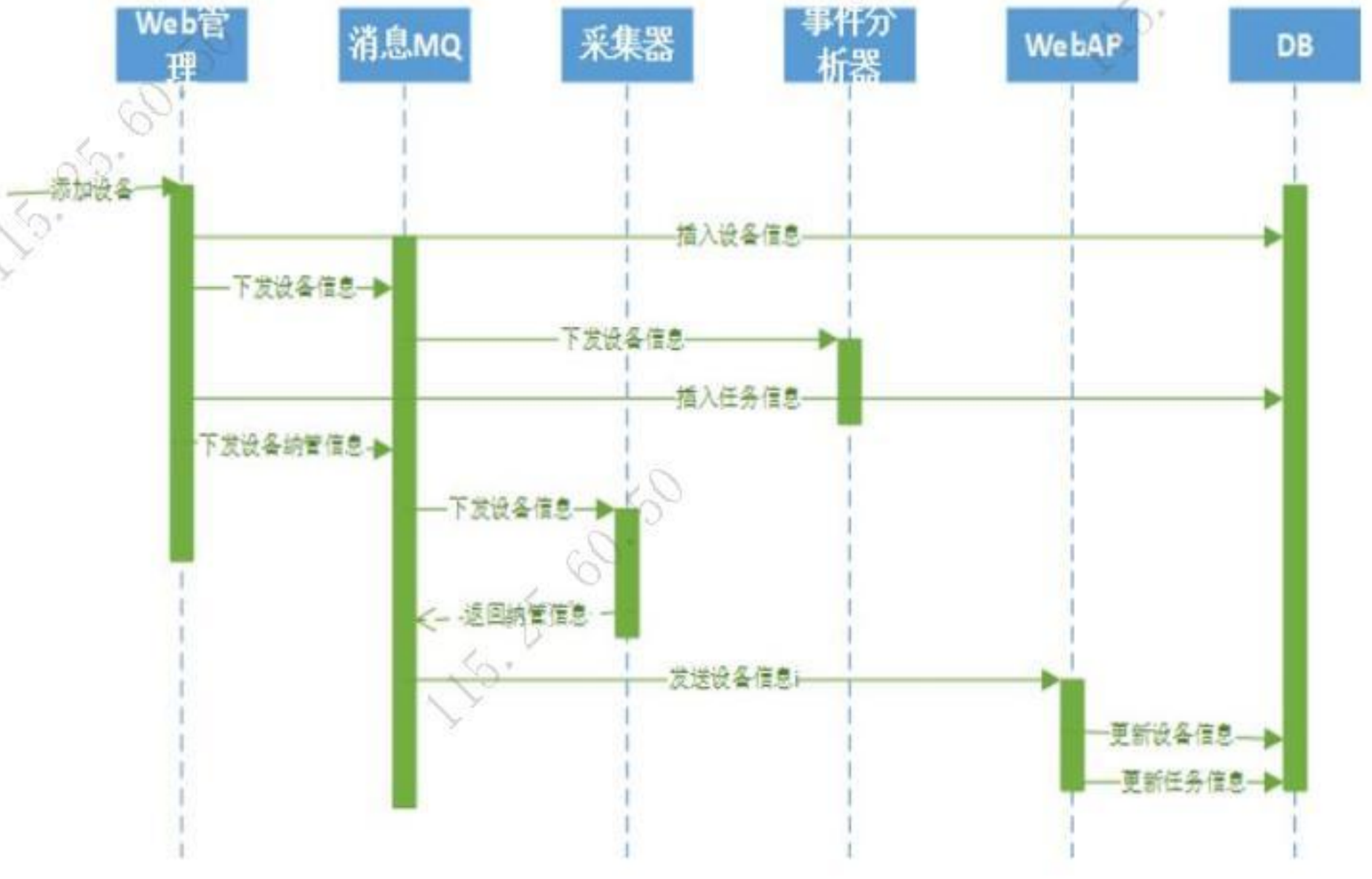


图 5-4 设备添加流程

Fig.5-4 Device addition process

设备管理提供网络设备纳管的入口，支持 SNMP 自动发现、手动添加、批量导入设备，记录设备 IP 地址、厂商、型号、位置、协议类型等基础信息，

维护社区串、端口号、采集间隔等设备采集参数用于设备维护。按地理区域、业务系统、设备类型进行分组展示，支持多层次分级结构，便于大规模网络设备统一管理。定义每台设备的 CPU 利用率、接口带宽、错误包数等监控项，将监控项与事件属性、聚合规则关联，构成采集与告警触发基础。实时更新在线、离线、故障等设备状态，支持设备启用/停用、重命名、删除操作。

本模块使用数据库存储设备信息表与分组关系，设备状态通过周期性心跳检测与 Ping 探测更新，支持 API 接口供采集模块调用设备参数。

图 5-4 以添加设备为例展示了设备管理的数据流与模块交互设计。

设备管理在执行设备添加操作时，通过多模块协同实现从前端指令到后台数据持久化的完整流程。用户在 Web 管理端提交“添加设备”请求后，前端将 IP 地址、厂商、协议版本等设备基本信息发送至消息中间件 MQ，由其负责将指令异步分发给后端采集器与事件分析模块。采集器接收到指令后，将设备信息插入设备列表并生成对应采集任务，同时将设备状态信息与任务配置写入数据库；事件分析器接收相应信息后，对设备的告警与运行状态进行注册和初始化。Web AP 模块在收到来自中间件或采集器的反馈后，更新设备状态及任务信息，并将最新状态同步至数据库，以实现前端的实时可视化展示。整个过程通过消息队列实现模块间异步通信与数据解耦，有效提高了系统在多设备并发管理场景下的响应效率与稳定性。



图 5-5 设备管理后端控制的类设计

Fig.5-5 Class Design for Backend Control of Device Management

图 5-5 为设备管理后端控制的类设计结构。类中定义了多组与设备管理相关的方法，涵盖设备信息查询、插入、更新、删除、状态检测与同步等多种功能。当前端界面发来设备管理请求，调用服务层和数据访问层实现设备

信息的增删改查、状态监控及关联操作。

系统实现的设备总览详细信息界面如图 5-6。

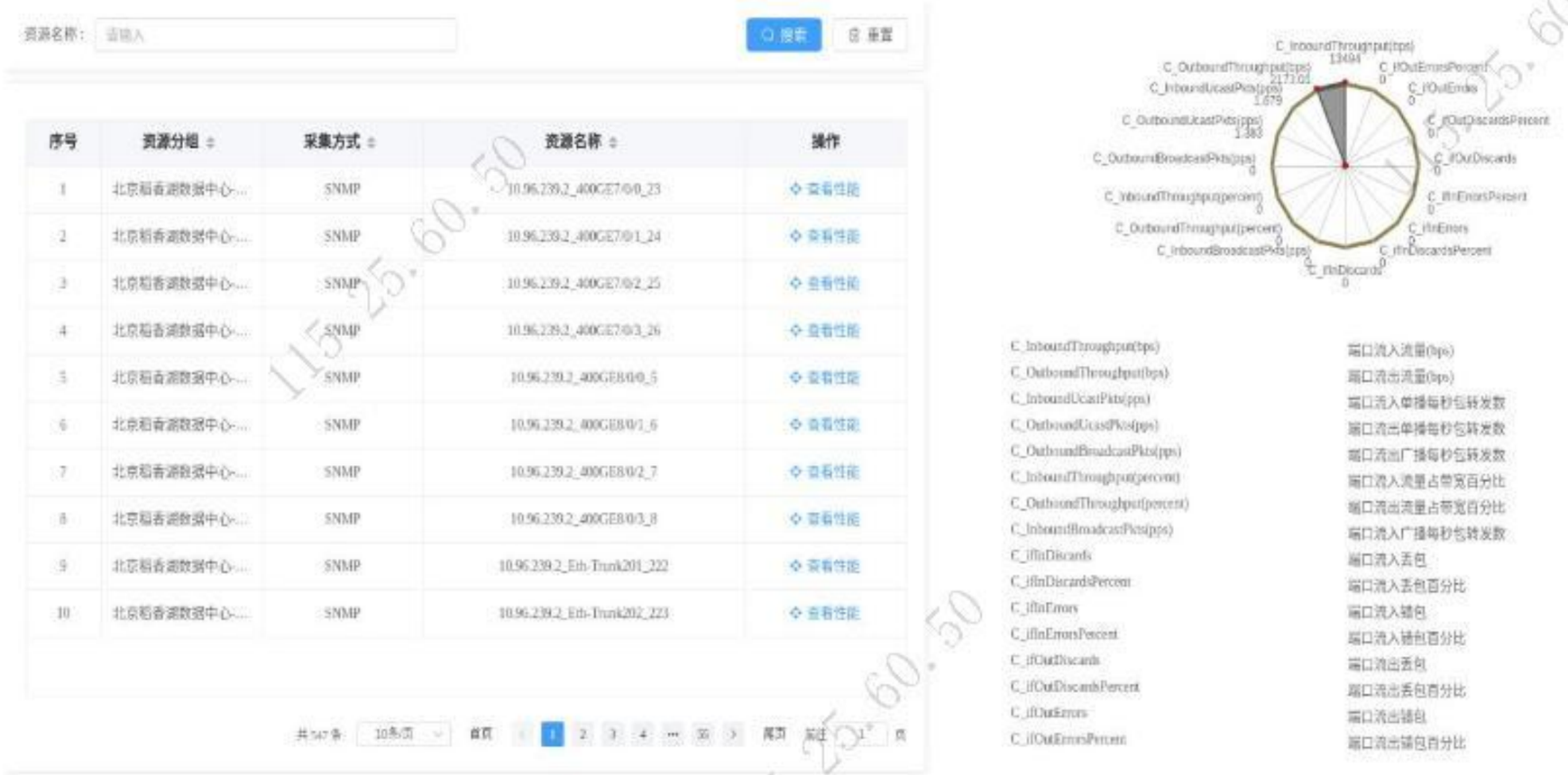


图 5-6 设备总览详细信息界面

Fig.5-6 Device Overview Detailed Information Interface

5.3.3 任务管理模块实现

任务管理模块是系统的调度中枢，负责采集、分析、聚合等任务的创建、分配、执行与监控，确保系统运转的高效性与有序性，可支持周期任务、即时任务、触发任务三种类型，用户可自定义任务名称、采集方式、目标设备、采集周期与超时策略，内置调度器根据设定规则定期执行任务，支持多线程并行执行、任务优先级控制与超时中断机制，提供任务运行日志、成功率统计与失败重试记录，并对延迟任务发出系统预警，任务间依赖通过调度策略自动协调。

本模块基于 Redis 实现分布式任务队列进行任务调度，任务日志存储于 Oracle 的任务执行表中，支持任务状态实时监控与历史查询。

性能数据采集任务流程逻辑如图 5-7 所示。主要通过 SNMP 等异构协议获取设备的配置信息与性能指标，并将采集结果存储至 Redis 集群。其运行流程包括配置加载、日志模块初始化、采集线程池创建、高可用监控及采集任务调度。系统首先加载配置文件，解析 ActiveMQ、Redis 集群、模块 ID 等参数；随后初始化日志记录模块，并建立 ActiveMQ 的生产者与消费者连接，实现采集任务的消息注册与资源更新。采集器为多线程采集，每个线程根据周期采集、模拟采集、特定 SNMP 采集、Ping 可达性测试、设备发现等任务类型选择相应的 SNMP 函数采集设备 OID 数据, 并使用 Lua 脚本公式计

算采集结果。计算后的性能数据被临时缓存至内存 map 结构中，并由定时线程异步写入 Redis，部分结果同时推送至 ActiveMQ 供上层模块实时使用。为提升系统可靠性，采集器引入了高可用处理机制：通过调用 Redis 中 GPP.DLOAD.DEADLINE 的群组 ID 字符串，定期更新进程时间戳，以区分主备节点，确保在主进程异常时自动接管任务。同时，系统还支持多指标采集公式扩展，通过 output_metric( ) 函数实现分项指标输出，使采集粒度更细、数据结构更灵活。

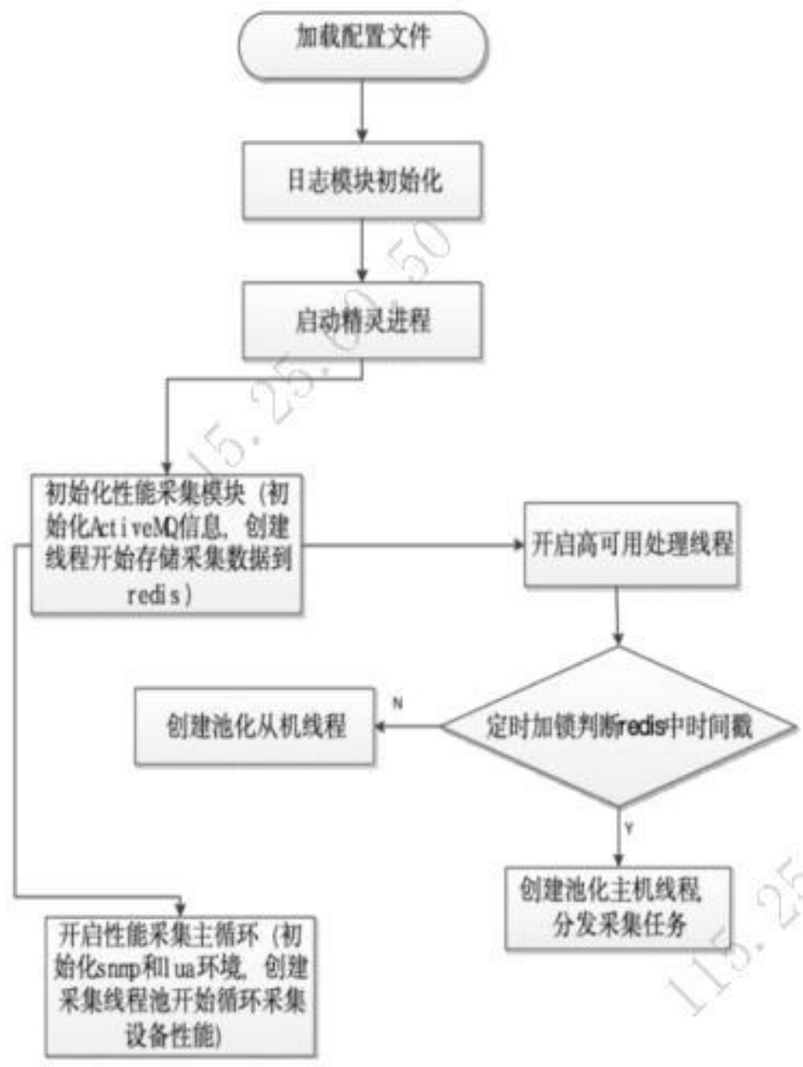


图 5-7 性能数据采集流程逻辑

Fig.5-7 Logic of Performance Data Acquisition Process

数据采集任务的数据库设计如图 5-8。

字段名	#	Type	类型修饰符	非空	缺省	注释
PING_MANAGE_ID	1	VARCHAR2(32)		[v]		
PEER_IP	2	VARCHAR2(64)		[ ]		对端IP地址
INTERVAL	3	NUMBER(38,0)		[ ]		采集间隔
PINGCOUNT	4	NUMBER(38,0)		[ ]		ping包数（1~15）
DATASIZE	5	NUMBER(38,0)		[ ]		Ping包大小（单位：bit），取值范围：0~65507
OPERTIMEOUT	6	NUMBER(38,0)		[ ]		超时秒数，取值范围：1~10
COLLECT_ID	7	VARCHAR2(32)		[ ]		采集器ID
CREATE_DATE	8	DATE		[ ]		创建时间
ISABLE	9	NUMBER(1,0)		[ ]		是否使用
STATUS	10	NUMBER(1,0)		[ ]		状态：0 未下发，1 已下发，2 修改后 未下发
DEVICE_ID	11	NUMBER(19,0)		[ ]		设备id
CI_ID	12	NUMBER(19,0)		[ ]		
CI_NAME	13	VARCHAR2(64)		[ ]		
L3TYPE_ID	14	NUMBER(38,0)		[ ]		mmdb三级分类id
DEV_INDEX	15	NUMBER(20,0)		[ ]		
TASK_STATUS	16	NUMBER(38,0)		[ ]		
TASK_STATUS_UPTIME	17	DATE		[ ]		
EVENT_PROP_ID	18	NUMBER(19,0)		[ ]		设备事件属性表id
DEVICE_TYPE	19	NUMBER(1,0)		[ ]		1:snmp设备,2:事件属性设备

图 5-8 数据采集任务的数据库设计

Fig.5-8 Database Design for Data Collection Tasks

端口流入流量指标和端口流出流量指标的采集任务结果如图 5-9 所示。

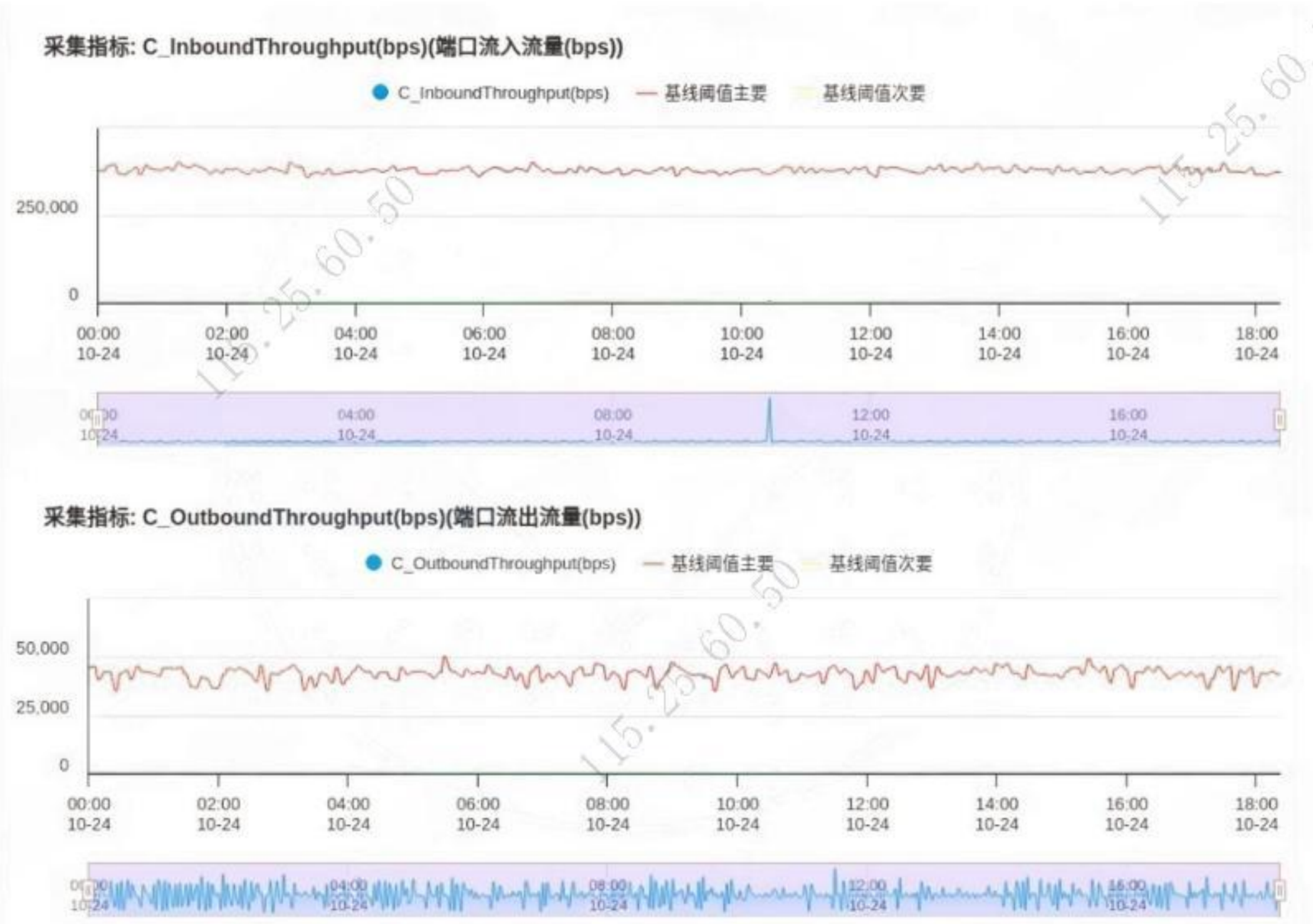


图 5-9 部分指标采集任务结果展示

Fig.5-9 Chart of Partial Results from Metric Collection Tasks

5.3.4 规则管理模块实现

规则管理模块用于定义、执行与管理性能数据的智能规则机制。规则计算基于 Pandas/Numpy 执行批量处理，调度由任务管理模块统一控制，结果写入 Oracle 供其他模块访问。

系统在事件处理逻辑中引入了延迟规则、压缩规则和聚合规则三种智能规则机制。三种智能规则机制的配置详情见表 5-1。

延迟规则用于对时间相关的事件进行缓存与关联分析，压缩规则通过识别高频重复告警，仅保留根因事件以抑制告警风暴。聚合规则则将集中发生的多指标、多设备事件进行汇总，生成统一的聚合事件以简化处理。这些规则由 rele_delay、comp_delay、gath_delay 等事件属性参数动态控制，支持灵活配置与自适应分析。

表 5-1 智能规则机制配置详情

Tab.5-1 Detailed Configuration of Intelligent Rule Mechanism

规则	事件属性	名称	说明
关联规则	rele_delay	关联时间	关联时间为 NULL 或 0：直接发送 关联时间 ＞1：延时处理，数值代表延时时间
	rele_status	关联状态	1：延时　0：延时结束（恢复）
	rele_key	分组标识	相同 Key 的事件将被缓存到一起
	rele_proc	关联处理	延时状态为 0 时触发关联处理 参数：在延时内的所有事件，包括恢复事件
压缩规则	comp_delay	压缩时间	压缩时间为 NULL 或 0：直接发送 压缩时间 ＞1：延时处理，数值代表延时时间
	comp_status	压缩状态	1：非压缩　0：压缩
	comp_key	分组标识	相同 Key 的事件将被缓存到一起
	comp_proc	压缩处理	压缩状态为 0 并延迟压缩时间后触发压缩处理 参数：在压缩事件前后压缩时间内的所有事件
聚合规则	gath_delay	聚合时间	聚合时间为 NULL 或 0：直接发送 聚合时间 ＞1：延时处理，数值代表延时时间
	gath_nums	聚合数量	聚合事件数量最大阈值
	gath_key	分组标识	相同 Key 的事件将被缓存到一起
	gath_proc	聚合处理	聚合事件数量大于 gath_nums 值时触发聚合处理 参数：在聚合时间内的所有事件

5.3.5 Ping 采集模块实现

Ping 采集模块用于检测设备连通性与网络质量，是系统实时状态监控的重要支撑。本模块使用 ICMP 协议检测主机可达性与延迟，并通过 RPing 实现双向链路检测，分析并记录链路带宽、抖动、时延、丢包率等指标，当 Ping 超时或丢包率超过阈值时自动触发告警。能够实时展示链路连通性图与性能曲线，并基于时序数据库记录 Ping 历史数据，用于稳定性评估。

采用多线程 ICMP 引擎并发检测，使用 Oracle 存储 Ping 结果数据，通过将告警接口与智能告警模块联动实现自动通知。

图 5-10 是系统实现的 Ping 采集界面。



图 5-10 Ping 数据采集界面

Fig.5-10 Ping Data Collection Interface

5.3.6 智能告警模块实现

智能告警模块通过本文第四章提出的基于知识推理的多算法融合自适应动态基线阈值方法动态调整多算法融合权重，实现阈值的自学习和实时优化，从而更合理地监测设备状态变化实现异常检测与实时预警。

支持在实时统计特征信息、设备上下文信息、历史监控知识与时间模式的基础上自动调整告警阈值，并支持日周期、周周期的自适应学习。将告警分为主要、次要两级，每级配置不同的处理流程与通知方式[71]。配置告警策略引擎，支持阈值、趋势、聚合异常多条件触发，同时支持对重复事件进行抑制，防止告警风暴的发生。对于短期内集中告警事件，可自动关联设备关系定位源问题辅助根因分析。支持邮件、短信、Web 推送等多种告警通知接口，可配置值班人员、通知模板与告警确认流程。

告警引擎采用异步任务架构，通知模块与第三方 API 集成实现多通道推送。

该模块中的事件分析器负责对网络设备产生的告警和故障信息进行解析与上报，实现网络事件的智能解析与处理，同时，事件分析器通过 KPI 资源同步机制实现采集公式与分项指标的一体化管理，当 KPI 配置变更时可自动同步资源下发，保持系统分析逻辑的一致性，流程逻辑如图 5-11 所示。通过 Syslog 和 SNMP Trap 协议实时接收事件信息，从 Redis 中读取协议数据后，

依据预定义解析规则进行格式化处理，并通过 ActiveMQ 异步发送至上层事件分析平台。

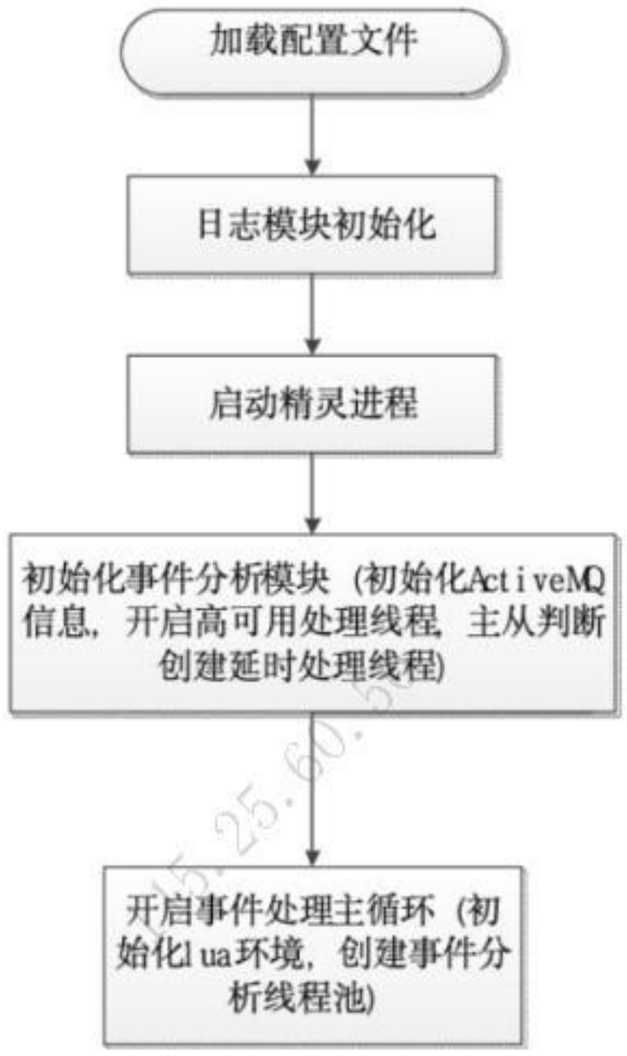


图 5-11 告警上报模块

Fig.5-11 Alarm Repoerting Module

5.3.7 日志管理模块实现

日志管理模块用于集中采集、存储与分析系统及设备日志，是运维分析与故障追踪的重要支撑模块。统一接收 Syslog 与 Trap 日志，记录平台运行事件及用户操作行为，支持关键字、时间、设备、级别等多条件组合的日志检索与过滤，通过生成日志类型分布图与事件趋势曲线完成日志统计与趋势分析，提供操作审计与安全合规检查功能。

本模块使用 ElasticSearch 实现全文索引与快速检索，日志集中存储于 Oracle 数据库，与性能报表模块联动生成日志统计报告。

5.3.8 性能报表模块实现

性能报表模块是系统的数据输出端，负责生成各类设备性能、健康度及合规性分析报告，为运维决策提供依据，自动生成 CPU、带宽、流量等指标的性能趋势曲线，按设备、区域、指标维度生成汇总统计表，通过计算设备健康评分与性能合规比率生成合规性与健康度报告。支持自定义模板与定时

任务自动生成报表，支持 PDF、Excel 多格式导出报表及邮件自动发送。

本模块的报表引擎基于 Jinja2 模板渲染，数据来源于聚合模块与 Oracle 数据库，结合任务调度模块实现自动化生成与分发。

5.4 系统测试

本节从功能、性能、安全性与高可用性、可扩展性与兼容性四个方面完成系统验证，测试环境与部署环境保持一致，测试过程均在控制条件下重复执行多次，以确保结果的准确性与可重复性。

5.4.1 功能测试

功能测试验证了系统功能实现是否正确，包括设备管理、任务调度、数据采集、事件告警与可视化展示五个方面。采用黑盒测试方法，通过前端操作与后端日志记录比对，验证系统行为是否符合设计规范。功能测试结果如表 5-2 所示。

表 5-2 功能测试结果

Tab.5-2 Results of Functional Testing

测试项	测试目标	测试步骤	预期结果	结果
设备管理	验证设备信息能否正确添加	在 Web 端输入设备 IP、厂商等信息	设备信息成功写入 Oracle 数据库	通过
任务管理	验证 SNMP 扫描设备可自动注册	执行 IP 段扫描任务	自动识别在线设备并显示状态	通过
数据采集	验证数据采集	下发采集任务并监测 Redis 缓存数据	Redis 中采集结果定时更新	通过
事件接收与告警	验证 Syslog 与 Trap 事件处理逻辑	模拟设备故障事件并观察告警记录	告警信息实时更新，推送至前端	通过
数据展示	验证数据可视化功能	选择设备接口流量指标生成曲线与报表	图表正确显示，报表可导出	通过

系统功能测试共执行 42 个测试用例，通过率 100%。设备信息管理、任务调度、采集执行与事件告警功能均运行正常，前后端数据同步及时，无数据丢失或接口异常。结果表明系统实现了多协议环境下的综合数据采集与可视化监控功能，整体业务逻辑符合设计预期。

5.4.2 非功能测试

5.4.2.1 性能测试

性能测试验证了系统在并发采集场景下的响应能力与数据处理能力。测试采用模拟器生成不同数量的设备与采集任务，并测量系统在高负载条件下的吞吐量、响应时间与资源占用率。

测试环境设置 5 秒采集周期、64 个 SNMP 采集线程、3 个 Redis 缓存节点、2 个 ActiveMQ 消费者，分别使用 100、500、1000、2000 台设备对系统吞吐量、平均响应时间、数据延迟和资源占用率等指标进行测试，结果如图 5-12 所示。

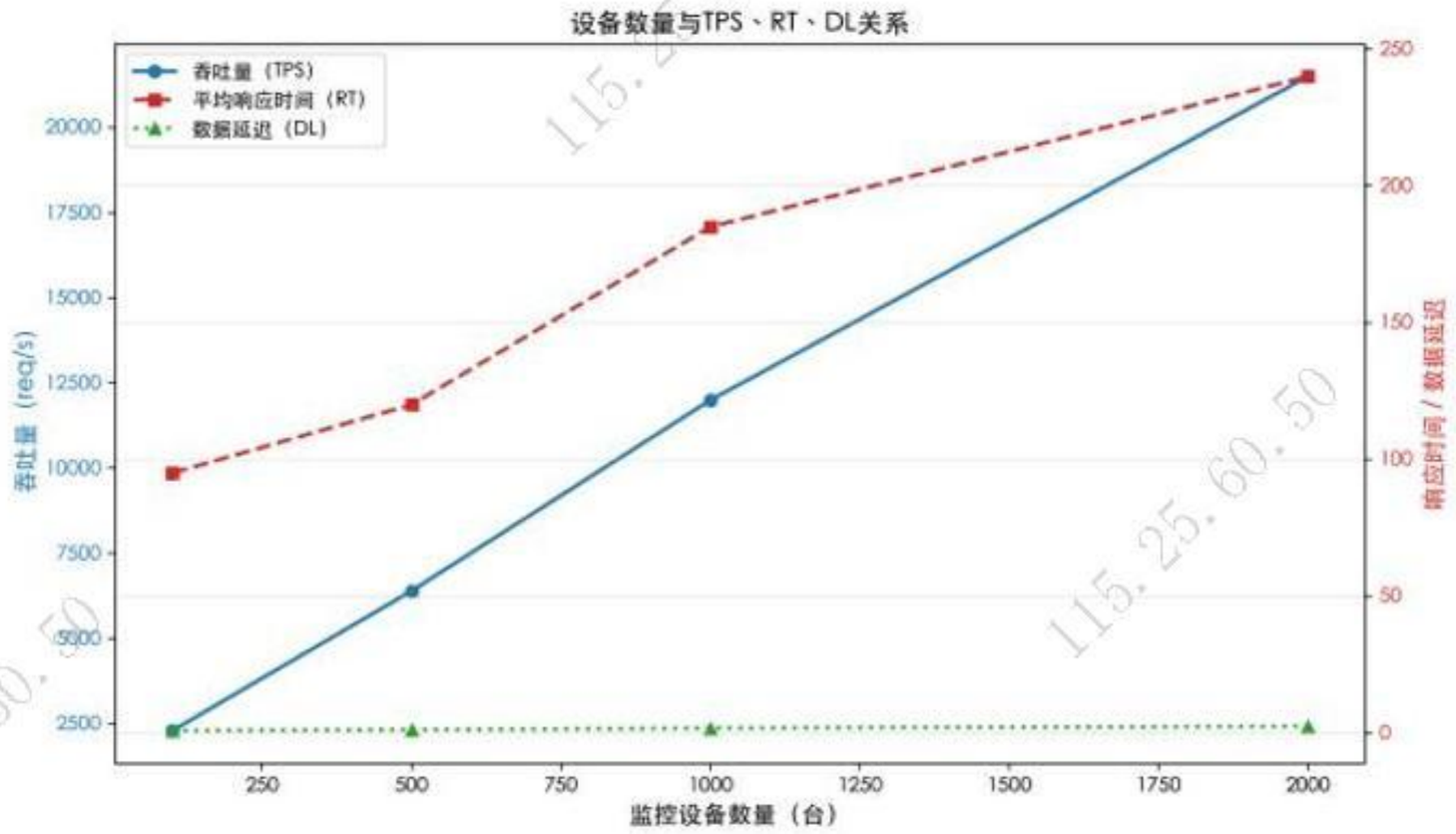


图 5-12 系统性能测试结果图

Fig.5-12 System Performance Test Results Diagram

当监控设备数量增加至 2000 台时，系统仍可维持平均 2.4 秒以内的数据延迟和 21500 次/秒的请求吞吐量，Redis 缓存命中率保持在 98%以上，前端实时展示延迟低于 2 秒，说明系统在大规模并发采集环境下性能表现稳定，具备良好的横向扩展能力与实时处理性能。

5.4.2.2 安全性与高可用性测试

安全性测试主要验证系统在访问控制、通信加密与数据完整性方面的安全防护能力。采用渗透测试与安全扫描相结合的方式，利用工具 Nessus 与 OWASP ZAP 对系统接口与传输通道扫描。安全性测试结果如表 5-3 所示。

表 5-3 安全性测试结果

Tab.5-3 Results of Security Testing

测试项	测试内容	测试方法	结果
用户认证	登录与权限控制	普通账户访问管理员	拒绝访问，权限控制有效
数据传输加密	HTTP 通信加密	抓包分析通信过程	全部为 HTTPS 加密通信
SQL 注入防护	输入过滤机制	构造异常 SQL 输入	防护有效，无异常响应
XSS 攻击防护	前端输入转义	插入恶意脚本测试	转义处理正确
操作日志追溯	系统操作记录	执行设备增删改查	日志记录完整，可追踪

系统具备完善的访问控制与通信加密机制，前后端交互数据均通过 HTTPS 加密传输，接口层参数过滤与转义处理有效，所有安全性测试项均通过，系统满足安全性要求。

高可用性测试用于验证系统在节点宕机、网络中断及任务异常情况下的自恢复与故障转移能力。采用人工触发与自动脚本结合的方式开展测试。高可用性测试结果如表 5-4 所示。

表 5-4 高可用性测试结果

Tab.5-4 Results of High Availability Testing

测试场景	测试过程	期望结果	实际结果
采集节点宕机	手动停止一台数据采集节点服务	备节点自动接管任务	10 秒内自动切换
ActiveMQ 服务异常	关闭消息队列服务	任务缓存等待恢复后重发	缓存数据未丢失，恢复后自动同步
Redis 节点故障	模拟主节点断开	副节点接管并自动同步	数据一致性保持
数据库断联	临时中断连接	进入缓存模式，恢复后同步	数据完整系统稳定

经过多轮高可用性测试，系统满足了服务故障自动检测、节点快速切换与数据一致性保障，任务恢复时间均低于 15 秒，能够有效证明系统在分布式

部署环境下具备良好的自恢复与容错能力，总体可用性达到 99.97%。

5.4.2.3 可扩展性与兼容性测试

为验证系统的扩展能力与跨平台兼容性，从节点扩展性测试与协议兼容性测试两个维度进行验证。测试目标是评估系统在设备规模扩展、节点增加及异构协议集成条件下的稳定性与适应性。

节点扩展性测试主要评估系统在增加节点数量和扩大规模后，整体性能及响应能力的变化趋势，测试通过逐步增加性能采集器与事件分析器节点数量，监测系统吞吐量、数据延迟与负载均衡效果。测试结果如图 5-13 所示。

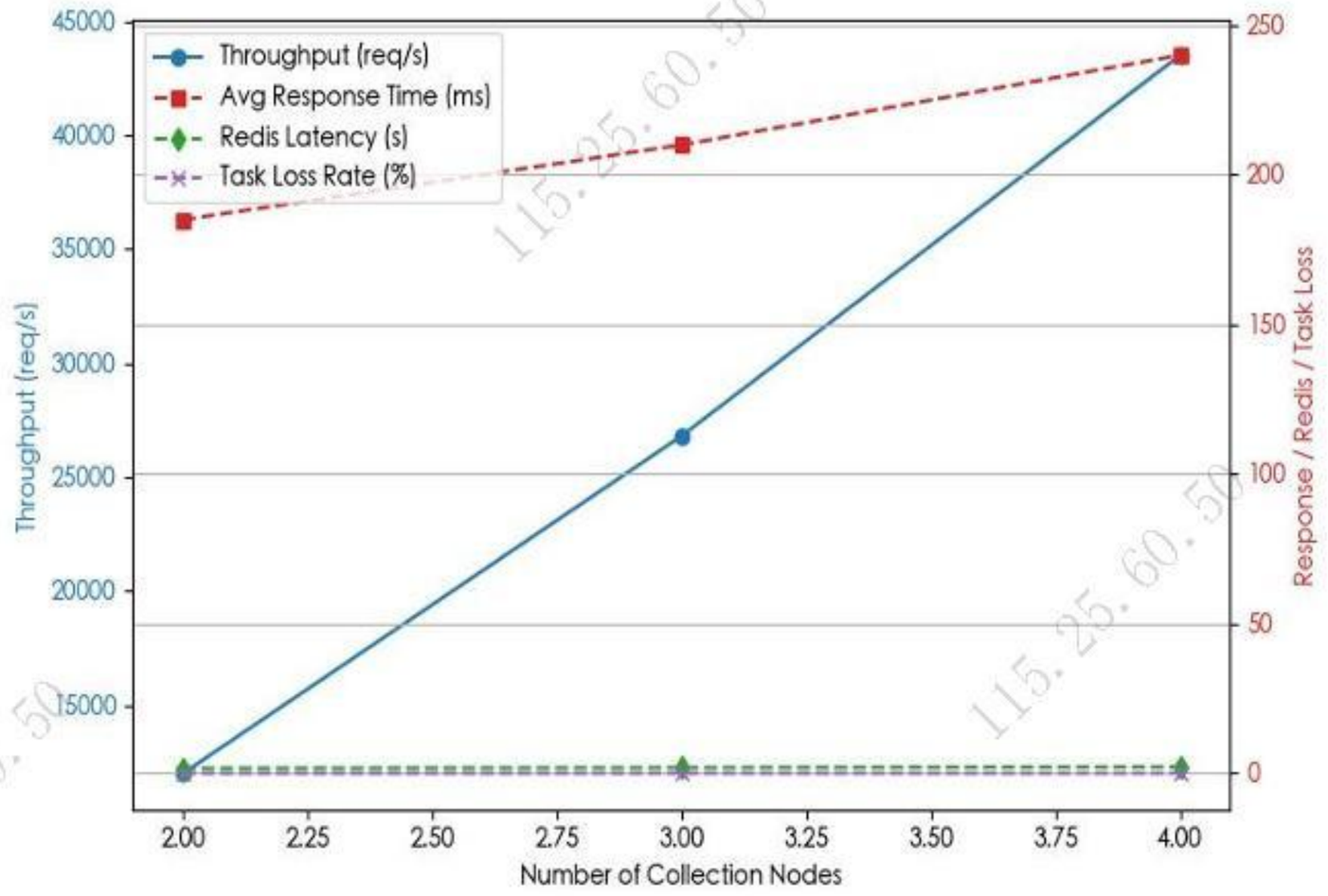


图 5-13 系统可扩展性测试结果图

Fig.5-13 System Scalability Test Results

随着采集节点数量的增加，系统吞吐量呈线性增长，任务丢失率保持在 0.03% 以下。当采集节点数量扩展至 4 个时，平均响应时间仅增加 29%，系统整体性能提升约 3.6 倍。这表明系统采用的分布式采集与消息队列异步传输机制能够有效支持横向扩展，具备良好的可扩展性。通过 Redis 与 ActiveMQ 的集群部署，系统在节点扩展后仍保持高并发处理能力与数据一致性。

协议兼容性测试用于验证系统对多种网络协议及不同厂商设备的适配能力。测试在混合环境下进行，包括 Cisco、Huawei、H3C、Juniper 等多类型设备，分别支持 SNMP、ICMP、Syslog、SNMP Trap 与 Telemetry 协议。兼容性测试结果如表 5-5 所示。

表 5-5 兼容性测试结果

Tab.5-5 Results of Compatibility Testing

协议类型	测试设备类型	功能验证	结果	说明
SNMP v2/v3	Huawei NE40E / Cisco 7600	参数采集、OID 解析	通过	支持多版本并行采集
ICMP	各类核心路由器	链路连通性测试	通过	延迟低于 5ms
Syslog	Huawei CE6857 / H3C S12500	日志接收与解析	通过	支持多格式日志分析
SNMP Trap	Juniper MX / Cisco ASR	告警事件捕获	通过	Trap 消息格式解析成功
Telemetry	Huawei VRP / Cisco IOS XR	实时流数据采集	通过	数据包丢失率低于 0.2%

系统能够在多厂商设备环境中稳定运行，支持五种主流协议的融合采集与数据处理。无论在 SNMPv2 与 SNMPv3 并存环境下，还是 Telemetry 高频推送模式下，系统均能自动识别协议版本并动态分配采集任务，具备跨设备兼容能力。

5.5 本章小结

本章围绕网络设备运行状态智能监控系统的实现与测试展开，全面介绍了系统的技术架构设计、功能模块实现以及系统测试结果。

介绍了系统的多层分布式技术架构设计，包括服务器层、操作系统层、数据库层、中间件层、开发框架层和前端框架层，通过标准化接口与消息中间件实现模块解耦与协作。

围绕系统的八个模块详细介绍了系统的功能实现。监控展示模块支持视图钻取、趋势分析与实时刷新，构建了多维可视化体系。设备管理模块实现了从设备纳管、分组、参数维护到状态监测的全生命周期管理。任务管理模块基于分布式调度与多线程执行机制，确保采集任务的高效分配与稳定执行。数据采集与 Ping 模块融合 SNMP、ICMP、Syslog、Telemetry 和 Trap 等多种网络协议，实现高并发、低延迟的数据采集与网络连通性检测。规则管理模块引入延迟、压缩与聚合三类智能规则机制，抑制冗余告警并提高事件处理的准确性与时效性。智能告警模块基于知识推理的多算法融合自适应动态基线阈值方法实现告警阈值的自学习与优化调整，具备多条件触发与多级通知能力。日志管理模块与性能报表模块分别承担系统运行追踪与数据输出功能。

本章还从功能、性能、安全性与高可用性、可扩展性与兼容性四个方面对系统进行了验证。功能测试涵盖设备管理、任务调度、数据采集、事件告

警及数据可视化，所有测试用例通过率达 100%，验证了系统已正确实现设计的所有功能。在大规模设备并发采集场景下，系统仍能保持高吞吐量、低数据延迟及高缓存命中率，表现出良好的横向扩展能力和实时处理性能。系统具备完善的访问控制、通信加密、SQL 注入与 XSS 防护机制，同时支持节点故障自动切换和任务恢复，系统总体可用性达到 99.97%。通过节点扩展与多协议环境测试，验证了系统在设备规模扩展和异构协议集成条件下的稳定性与适应性，说明系统具备良好的扩展能力和兼容能力。

综上，本章系统性展示了网络设备运行状态智能监控系统从技术架构设计、模块实现到功能与非功能验证的全过程，验证了系统在高并发、多协议融合、智能告警及可视化展示等方面的有效性。

6 结论与展望

6.1 总结

本文围绕数据中心网络设备数量日趋增大背景下的运维需求，针对当前网络设备监控过程中普遍存在的协议异构导致的数据采集效率低、静态阈值引发的误报与漏报率高、告警机制智能化不足以及系统扩展性与可用性有限等关键问题，设计并实现了网络设备运行状态智能监控系统。本研究从需求分析出发，结合多协议融合与智能基线建模两条技术主线，形成了完整的体系架构、方法创新与实验验证流程，构建了可落地的智能化网络监控解决方案。主要研究成果和创新点总结如下。

提出了基于异构协议融合的网络设备数据采集方法。在分析传统 SNMP、ICMP、Syslog、SNMP Trap 和 Telemetry 等协议在实时性、数据精度与兼容性方面的特性差异基础上，本文设计了一种模块化、分层化的异构协议融合框架。该框架包括数据采集层、数据融合与处理层以及智能调度与协议选择层三个核心模块，通过协议间的协同与动态选择，实现多源数据的高效采集与融合处理。实验结果表明，异构协议融合机制在数据采集效率、实时性和准确率方面均显著优于单一协议方案，其中平均采集延迟降低约 48.7%，准确率提升约 8.9%。该方法显著提升了监控数据的时效性、完整性与可靠性，为后续智能分析与告警提供了统一、稳定的数据基础。

设计了基于知识推理的多算法融合自适应动态基线阈值算法，针对固定阈值误报与漏报率高的问题，本文提出了一种融合统计分析、平滑拟合与补偿修正等多算法的动态基线生成方法。通过引入知识推理机制，系统利用大模型推理与规则约束对多算法权重进行自适应优化，实现阈值的动态调整与智能更新。实验结果表明，该方法较传统的多算法融合方法 F1 提升约 9.5%、误报率降低约 45%，有效提升了异常检测的敏感度与准确性，减少了误报与漏报现象，使监控系统能够在复杂波动与多负载场景下保持稳定运行，显著提升了系统的智能化水平。

完成系统实现与综合验证。本文基于前面提出的数据采集和动态基线方法实现了系统原型，并在实验环境中从功能性、性能、安全性与高可用性、可扩展性与兼容性等方面展开测试，验证了系统在多协议协同采集、动态阈值检测、智能告警和可视化展示等方面的有效性与实用价值。

综上所述，本文通过异构协议融合的数据采集机制与基于知识推理的多算法融合动态阈值模型两条主线的联合创新，构建了一套集智能化、可扩展性与高可靠性于一体的网络设备运行状态监控系统。该系统不仅在理论上完善了多源异构数据采集与智能阈值建模的研究体系，也在工程实践中为网络运维智能化的实现提供了可行的技术路径与验证依据，对未来大规模网络系统的自主监测与智能运维具有重要的研究价值与推广意义。

6.2 展望

尽管本文在网络设备运行状态智能监控体系的设计与实现方面取得了较为系统和完整的研究成果，但仍存在一些有待深入研究和完善的地方。

动态基线模型多场景泛化。本文在金融网络与数据中心场景中验证了 KR-MFA-DBT 模型的有效性，模型能够在周期性波动与高负载条件下保持较高检测准确率。但由于不同行业在指标特征、时序分布及异常模式上存在显著差异，模型在跨行业或跨系统迁移时仍需人工干预或重新参数校准。后续工作可在模型中引入领域自适应机制与元学习策略，使系统能够在新场景下快速适配指标变化并自动完成参数迁移，从而增强模型普适性与泛化能力。

运维闭环联动与更完善的自动化响应。目前系统已实现从数据采集、指标分析到智能告警与可视化展示的闭环，但未与自动化运维平台实现深度集成。告警后的处置与策略优化仍依赖人工干预，难以实现端到端的智能修复与策略反馈。后续可通过引入事件驱动机制将监控结果与自动化运维流程结合，实现监控、分析、决策、执行的完整运维闭环，实现真正意义上的自主决策与自愈化运维。

参考文献

[1] Yaseen N. From Counters to Telemetry: A Survey of Programmable Network-Wide Monitoring[J]. Network, 2025, 5(3): 38.

[2] Zhou Y, Zhang Y, Yu M, et al. Evolvable network telemetry at facebook[C]//19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22). 2022: 961-975.

[3] Wang F, Jiang Y, Zhang R, et al. A survey of deep anomaly detection in multivariate time series: taxonomy, applications, and directions[J]. Sensors (Basel, Switzerland), 2025, 25(1): 190.

[4] El-Shamy A M, El-Fishawy N A, Attiya G, et al. Anomaly detection and bottleneck identification of the distributed application in cloud data center using software–defined networking[J]. Egyptian informatics journal, 2021, 22(4): 417-432.

[5] Ivanov I. Comparing the performance of SNMP to network telemetry streaming with gRPC/GPB[C]//53rd Int. Sci. Conf. Inf. Commun. Energy Syst. Technol. 2018: 175-178.

[6] Case J D, Fedor M, Schoffstall M L, et al. Simple network management protocol (SNMP)[R]. 1989.

[7] Postel, Jon. Internet control message protocol. No. rfc792. 1981.

[8] Case J, Harrington D, Presuhn R, et al. Message processing and dispatching for the simple network management protocol (snmp)[R]. 2002.

[9] Gerhards, Rainer. The Syslog protocol. No. rfc5424. 2009.

[10] Rose, Marshall T. Convention for defining traps for use with the SNMP. No. rfc1215. 1991.

[11] Rodgers A R. Recent telemetry technology[M]//Radio tracking and animal populations. Academic Press, 2001: 79-121.

[12] J. McCown, et al., Network Telemetry Framework, RFC 9232, IETF, 2021.

[13] Zhou Y, Zhang Y, Yu M, et al. Evolvable network telemetry at facebook[C]//19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22). 2022: 961-975.

[14] Martinez-Casanueva I D, Gonzalez-Sanchez D, Bellido L, et al. Toward

building a semantic network inventory for model-driven telemetry[J]. IEEE Communications Magazine, 2023, 61(3): 60-66.

[15] Yu M. Network telemetry: towards a top-down approach[J]. ACM SIGCOMM Computer Communication Review, 2019, 49(1): 11-17.

[16] Landau-Feibish S, Liu Z, Rexford J. Compact Data Structures for Network Telemetry[J]. ACM Computing Surveys, 2025, 57(8): 1-31.

[17] Raptis T P, Passarella A. A survey on networked data streaming with apache kafka[J]. IEEE access, 2023, 11: 85333-85350.

[18] Oliveira R, Pedrosa T, Rufino J, et al. Parameterization and Performance Analysis of a Scalable, near Real-Time Packet Capturing Platform[J]. Systems, 2024, 12(4): 126.

[19] Hellerstein J L, Zhang F, Shahabuddin P. A statistical approach to predictive detection[J]. Computer Networks, 2001, 35(1): 77-95.

[20] Hajji H. Baselining network traffic and online faults detection[C]//IEEE International Conference on Communications, 2003. ICC'03. IEEE, 2003, 1: 301-308.

[21] Breitgand D, Goldstein M, Henis E, et al. Performance management via adaptive thresholds with separate control of false positive and false negative errors[C]//2009 IFIP/IEEE International Symposium on Integrated Network Management. IEEE, 2009: 195-202.

[22] Breitgand D, Henis E, Shehory O. Automated and adaptive threshold setting: Enabling technology for autonomy and self-management[C]//Second International Conference on Autonomic Computing (ICAC'05). IEEE, 2005: 204-215.

[23] Hajji H. Statistical analysis of network traffic for adaptive faults detection[J]. IEEE Transactions on Neural Networks, 2005, 16(5): 1053-1063.

[24] Hu K, Wang Z, Yin B. Baselining network-wide traffic by time-frequency constrained Stable Principal Component Pursuit[J]. AEU-International Journal of Electronics and Communications, 2014, 68(8): 756-762.

[25] Yu H, Yang W, Cui B, et al. Renyi entropy-driven network traffic anomaly detection with dynamic threshold[J]. Cybersecurity, 2024, 7(1): 64.

[26] Yu Y, Zhu Y, Li S, et al. Time series outlier detection based on sliding

window prediction[J]. Mathematical problems in Engineering, 2014, 2014(1): 879736.

[27] Isaac E R H P, Sharma A. Adaptive thresholding heuristic for KPI anomaly detection[C]//2024 16th International Conference on COMmunication Systems & NETworkS (COMSNETS). IEEE, 2024: 737-741.

[28] Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning[J]. nature, 2015, 518(7540): 529-533.

[29] Yu H, Yang W, Cui B, et al. Renyi entropy-driven network traffic anomaly detection with dynamic threshold[J]. Cybersecurity, 2024, 7(1): 64.

[30] Russell-Gilbert A, Sommers A, Thompson A, et al. Aad-llm: Adaptive anomaly detection using large language models[C]//2024 IEEE International Conference on Big Data (BigData). IEEE, 2024: 4194-4203.

[31] Yu Y C, Ouyang Y C, Lin C A. Trip-llm: A tri-branch patch-wise large language model framework for time-series anomaly detection[J]. IEEE Access, 2025.

[32] 冯万贵. IT 运维服务质量改进与 IT 服务管理体系建设[J]. 价值工程, 2013, 32(23): 168-171.

[33] 刘佩琪. 依托安全运行平台 打造高效运维体系——以黄埔海关信息系统运维监控中心建设为例[J]. 上海海关学院学报, 2011, 32(2): 91-97.

[34] 常小燕, 宋长青, 张福岭. 计算机网络实验室建设方案设计探索[J]. 科技广场, 2007 (3): 239-241.

[35] 王焘, 张文博, 徐继伟, 等. 云环境下基于统计监测的分布式软件系统故障检测技术研究[J]. 计算机学报, 2017, 2: 397-413.

[36] González I, Calderón A J, Portalo J M. Innovative multi-layered architecture for heterogeneous automation and monitoring systems: Application case of a photovoltaic smart microgrid[J]. Sustainability, 2021, 13(4): 2234.

[37] Cabrera O, Oriol M, Franch X, et al. A context-aware monitoring architecture for supporting system adaptation and reconfiguration[J]. Computing, 2021, 103(8): 1621-1655.

[38] 刘巍, 黄曌, 李鹏, 等. 面向智能配电网的大数据统一支撑平台体系与构架[J]. 电工技术学报, 2014 (S1): 486-491.

[39] 吴娟, 孙俊虎, 闫印强. 全流量日志 AI 智能分析系统[J]. 信息安全与通信保密, 2020 (S01): 144-148.

[40] 崔峰, 郭刚, 饶伟, 等. 基于 LSTM 的云环境异常智能检测方法研究[J]. 铁路计算机应用, 2020, 29(6): 44-48, 53.

[41] 杨超峰, 刘庆云, 刘利军. 多层次网络安全强审计模型 BrosaAudit[J]. 计算机工程, 2006, 32(10): 159-160.

[42] Schonwalder J, Pras A, Harvan M, et al. SNMP traffic analysis: Approaches, tools, and first results[C]//2007 10th IFIP/IEEE International Symposium on Integrated Network Management. IEEE, 2007: 323-332.

[43] Martinez-Casanueva I D, Gonzalez-Sanchez D, Bellido L, et al. Toward building a semantic network inventory for model-driven telemetry[J]. IEEE Communications Magazine, 2023, 61(3): 60-66.

[44] 陈明兵, 刘知贵. 用于网络流量异常检测的数据采集方法的研究[J]. 电脑知识与技术: 学术版, 2009, 5(2): 822-824.

[45] Abane A, Battou A, Amlou A, et al. A Data Collection Platform for Network Management[C]//2023 20th ACS/IEEE International Conference on Computer Systems and Applications (AICCSA). IEEE, 2023: 1-7.

[46] Zhao Z, Guo H, Wang Y. A multi-information fusion anomaly detection model based on convolutional neural networks and AutoEncoder[J]. Scientific Reports, 2024, 14(1): 16147.

[47] Bolla R, Comi P M, Repetto M. A distributed cyber-security framework for heterogeneous environments[C]//CEUR WORKSHOP PROCEEDINGS. CEUR-WS, 2018: 1-6.

[48] Abba Ari A A, Aziz H A, Njoya A N, et al. Data collection in IoT networks: Architecture, solutions, protocols and challenges[J]. IET Wireless Sensor Systems, 2024, 14(4): 85-110.

[49] 肖安. 大规模容器云平台稳定性闭环解决方案的设计与实现[D]. 浙江大学, 2020.

[50] 陈亚. 网络安全态势感知技术研究[J]. 江苏科技信息, 2019, 36(3): 38-41.

[51] Cheng H, Feng D, Shi X, et al. Data quality analysis and cleaning strategy for wireless sensor networks[J]. EURASIP Journal on Wireless

Communications and Networking, 2018, 2018(1): 61.

[52] Makonyi K, Abrahamsson H, Henriksson D, et al. On the use of streaming telemetry data for network health monitoring and anomaly detection[C]//Swedish National Computer Networking and Cloud Computing Workshop (SNCNW). 2024.

[53] Tsanousa A, Bektsis E, Kyriakopoulos C, et al. A review of multisensor data fusion solutions in smart manufacturing: Systems and trends[J]. Sensors, 2022, 22(5): 1734.

[54] Simakovic M, Cica Z, Drajic D. Big-data platform for performance monitoring of telecom-service-provider networks[J]. Electronics, 2022, 11(14): 2224.

[55] Sun P, Zhao B, Li X. Research on multi-source heterogeneous data fusion method of substation based on cloud edge collaboration and AI technology[J]. Discover Applied Sciences, 2025, 7(4): 262.

[56] 张强, 张伯明, 李鹏. 智能电网调度控制架构和概念发展述评[J]. 电力自动化设备, 2010, 12.

[57] Schmidl S, Wenig P, Papenbrock T. Anomaly detection in time series: a comprehensive evaluation[J]. Proceedings of the VLDB Endowment, 2022, 15(9): 1779-1797.

[58] Lei L, Kou L, Zhan X, et al. An anomaly detection algorithm based on ensemble learning for 5G environment[J]. Sensors, 2022, 22(19): 7436.

[59] Jeffrey N, Tan Q, Villar J R. Using ensemble learning for anomaly detection in cyber – physical systems[J]. Electronics, 2024, 13(7): 1391.

[60] Chang Y, Wang X, Wang J, et al. A survey on evaluation of large language models[J]. ACM transactions on intelligent systems and technology, 2024, 15(3): 1-45.

[61] Zhou A, Xu X, Raghunathan R, et al. KnowGraph: Knowledge-Enabled Anomaly Detection via Logical Reasoning on Graph Data[C]//Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. 2024: 168-182.

[62] Su K, Wu J, Gu D, et al. An adaptive deep ensemble learning method for dynamic evolving diagnostic task scenarios[J]. Diagnostics, 2021, 11(12):

2288.

[63] Ekle O A, Eberle W. Anomaly detection in dynamic graphs: A comprehensive survey[J]. ACM Transactions on Knowledge Discovery from Data, 2024, 18(8): 1-44.

[64] Lehmann R. 3 σ -rule for outlier detection from the viewpoint of geodetic adjustment[J]. Journal of Surveying Engineering, 2013, 139(4): 157-165.

[65] Burgetová I, Matoušek P, Ryšavý O. Anomaly detection of ICS communication using statistical models[C]//2021 17th International Conference on Network and Service Management (CNSM). IEEE, 2021: 166-172.

[66] Prasad Y A S, Krishna G R. Statistical anomaly detection technique for real time datasets[J]. International Journal of Computer Trends and Technology (IJCTT), 2013, 6(2): 89-94.

[67] Xu Y, Luo M, Li T, et al. ECG signal de-noising and baseline wander correction based on CEEMDAN and wavelet threshold[J]. Sensors, 2017, 17(12): 2754.

[68] Gan F, Ruan G, Mo J. Baseline correction by improved iterative polynomial fitting with automatic threshold[J]. Chemometrics and Intelligent Laboratory Systems, 2006, 82(1-2): 59-65.

[69] Gilliard E, Liu J, Aliyu A A. Knowledge graph reasoning for cyber attack detection[J]. IET Communications, 2024, 18(4): 297-308.

[70] 王庚, 张景辉, 吴娜. 网络安全态势预测方法的应用研究[J]. 计算机仿真, 2012, 29(2): 98-101.

[71] 陈建宏. 基于深度学习的智能监控平台设计[D]. 北京交通大学, 2019.

致　谢

时光荏苒，转眼间我的研究生学习即将结束。回顾这段学习的历程，我深感收获颇丰，也感慨万千，从研招考试时面临疫情防控解除时的担忧，到初入校园时候的兴奋，这两年多的时光里我和北科大一同走过春夏秋冬，一道经历风雨彩虹，一个个瞬间定格在这不平凡的时光里。经过研究生涯的学习和研究，不仅让我在专业知识与理论学习方面得到了系统性的提升，也让我在工作实践与学术研究的结合中积累了宝贵的经验。在论文的研究与撰写过程中，我得到了许多师长、同事和朋友的关心与支持，也在此谨致以最诚挚的谢意。

首先，我要衷心感谢我的导师——周贤伟老师。周老师以严谨的治学态度、深厚的专业素养和耐心细致的指导，在我整个研究过程中给予了悉心的帮助。从论文选题、研究思路的确立，到系统设计、实验验证及论文撰写的各个阶段，周老师都提供了宝贵的建议和无私的指导。在学术研究之外，周老师还为我创造了多次学习与交流的机会，使我得以开阔视野、增长见识，并在科研规范与职业素养方面受益匪浅。周老师的谆谆教诲将成为我今后工作与学习道路上的宝贵财富。

其次，我要感谢单位的领导与同事们，感谢他们在我攻读研究生期间给予的理解与支持，在繁忙的工作之余，他们为我提供了良好的研究环境和实践条件，使我能够将理论研究与实际工作紧密结合，提升了研究的现实意义。同时，也特别感谢那些与我同出师门的同事们，他们在各自岗位上取得的优异成绩也不断激励着我前行，也让我深刻体会到持续学习与终身进步的重要价值。

最后，感谢我的家人们，感谢你们一直以来对我的包容与支持，正因为有你们在背后为我默默的付出，不断地鼓励与陪伴我，我才能坚持完成学业并顺利完成论文写作。

再次向所有在我学习与研究过程中给予过指导、帮助和鼓励的人士表示由衷的感谢!

作者简历及在学研究成果

一、主要教育经历/工作经历

起止年月	学习或工作单位	备注
2014 年 9 月至 2018 年 6 月	在东北师范大学计算机科学与技术专业攻读学士学位	
2018 年 7 月至今	在中国建设银行运营数据中心工作	

二、在学期间从事的科研工作

无

三、在学期间所获的科研奖励

无

四、在学期间发表的论文

无

独创性说明

本人声明所呈交的论文是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京科技大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作 者 签 名：李根 日期：2025 年 12 月 20 日

关于论文使用授权的说明

本人完全了解北京科技大学有关保留、使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

(保密的论文在解密后应遵循此规定)

签名：李根　导师签名：周学书　日期：2025年12月20日